ESE PYQ Solutions

Artificial Intelligence — Semester VI (TY B.Tech CE)

K. J. Somaiya College of Engineering

Exam-ready answers for Jan 2021 – Apr 2025 papers

Papers covered:

1. Jan 25 – Apr 25
2. Jan 24 – Apr 24
3. Jan 23 – May 23
4. May 22 – Jun 22 (Sections A & B + Addendum)
5. May 21 – Jun 21

PAPER 1 : JAN 2025 – APR 2025

Q1 — Solve any FOUR (20 marks)

Q1 i) Discuss any five limitations of AI. (5)

Artificial Intelligence, despite enormous progress, has the following key limitations:

- **Lack of common sense:** AI systems lack the everyday reasoning that humans take for granted. A self-driving car may not realise that a child running after a ball might cross the road, unless explicitly programmed.
- **No genuine creativity / no original thinking:** AI can only generate combinations of patterns seen in training data. It does not invent new paradigms the way humans do (e.g. relativity, abstract art).
- **Data dependency and bias:** AI requires vast labelled data. If data is biased, incomplete, or noisy, the model becomes biased (e.g. facial recognition working poorly on darker skin tones).
- **No emotions / no empathy:** AI cannot truly understand human emotions, sarcasm, or cultural context. A chatbot can recognise sentiment but cannot feel.
- **Explainability and trust (black-box):** Deep learning models are opaque. In high-stakes domains (medical diagnosis, judiciary) the absence of explanation prevents adoption.
- **High computational and energy cost:** Training models like GPT requires enormous GPUs/TPUs and power, which is environmentally and economically costly.
- **Ethical and security concerns:** AI can be misused (deepfakes, autonomous weapons), and is vulnerable to adversarial attacks.

Q1 ii) Define Heuristic. Manhattan distance for 8-puzzle. Effect of choice of heuristic. (1+2+2)

Heuristic: A heuristic h(n) is a problem-specific function that estimates the cost of the cheapest path from the current node n to a goal. It is used by informed search algorithms (e.g. A*, Best-First) to guide the search toward the goal more efficiently than blind search.

Manhattan distance for 8-puzzle: For each tile (except the blank), Manhattan distance is the sum of the horizontal and vertical moves required to bring it from its current position to its goal position. The heuristic value h(n) for the state is the sum of Manhattan distances of all misplaced tiles.

Example — if tile '1' is at position (2,2) and its goal is (0,0), its contribution = |2-0| + |2-0| = 4. Add similar values for tiles 2 to 8 to get h(n).

Effect of choice of heuristic on efficiency:

- A **more informed (closer-to-true-cost)** heuristic such as Manhattan dominates a weaker one like 'Misplaced Tiles'. The more informed heuristic expands fewer nodes and reaches the goal faster.
- If the heuristic is **admissible** (never overestimates), A* gives an optimal solution.
- If the heuristic is **consistent (monotone)**, A* never re-expands nodes — saving memory and time.
- A poorly chosen, non-admissible heuristic may lead to a non-optimal (sub-optimal) solution. A zero heuristic reduces A* to Uniform Cost Search.

Q1 iii) PEAS analysis — Garden watering system. (5)

PEAS = Performance measure, Environment, Actuators, Sensors. For an agent that waters plants when soil is dry and stops when wet:

Component	Description for Garden Watering System

Performance Measure	• Plants kept healthy (soil moisture in optimal range) • Minimum water wastage • No over-watering / root rot • Low energy cost / pump runtime • Quick response to dryness
Environment	• Garden / soil bed • Plants of various species • Weather (sun, rain, temperature, humidity) • Water source (tank / supply line) • Time of day
Actuators	• Solenoid valve / motorised tap (ON-OFF) • Water pump / sprinkler • Drip irrigation lines • Status LEDs or buzzer for alerts
Sensors	• Soil-moisture sensor (capacitive / resistive) • Temperature & humidity sensor (DHT22) • Rain detector • Water level sensor in tank • Light / time-of-day sensor (RTC)

Q1 iv) Problem formulation for N-Queens problem. (5)

The N-Queens problem: place N queens on an N×N chessboard so that no two queens attack each other (no shared row, column, or diagonal).

1. Initial State: An empty N×N board (no queens placed). Represented as an empty list / array of size N.

2. State Representation: An array Q[1..N] where Q[i] = column of the queen placed in row i. Length = number of queens placed so far.

3. Actions / Successor Function: From a state with k queens placed, generate successor states by placing the (k+1)-th queen in any column 1..N of row k+1 such that it does not attack any previously placed queen.

4. Goal Test: A state is a goal if all N queens are placed and no two attack each other. Equivalently: length(Q) = N and for every $i \neq j$: $Q[i] \neq Q[j]$ AND $|Q[i] - Q[j]| \neq |i - j|$.

5. Path Cost: Not relevant — only the final configuration matters. Usually taken as 1 per placement (so cost = N for any solution).

Outcomes / Approach: Solved using DFS with backtracking; CSP formulation also works (variables = rows, domains = columns, constraints = non-attack). For N = 8 there are 92 distinct solutions (12 unique up to symmetry).

Q1 v) Adversarial Search — Define Max, Min, alpha. Apply MAXMIN to tree. (1+1+1+2)

MAX: The player trying to **maximise** the value at a node. At a MAX node, value = maximum of the values of its children.

MIN: The opponent trying to **minimise** the value. At a MIN node, value = minimum of the values of its children.

Alpha (α): The **best (highest) value** that MAX can guarantee along the current path so far. Used in α-β pruning. Initialised to $-\infty$ and updated at MAX nodes.

MAX-MIN application on given tree: Leaves: D=3,5 ; E=6,9 ; F=2,0 ; G=1.

Since D and E are MAX nodes: D = max(3,5) = 5 ; E = max(6,9) = 9. F and G are MAX nodes too: F = max(2,0) = 2 ; G = 1 (only one child shown).

Then B is MIN: B = min(D, E) = min(5, 9) = 5. C is MIN: C = min(F, G) = min(2, 1) = 1. Root A is MAX: A = max(B, C) = max(5, 1) = **5**.

Value at root A = 5. Optimal first move for MAX → go to B.

Q1 vi) Difference between AI program and Expert System. Significance. (5)

Aspect	AI Program (general)	Expert System
Scope	Solves a general problem class	Solves problems of one specific narrow domain (e.g. medical di
Knowledge	Algorithmic logic + data	Explicit knowledge base of domain rules captured from human e
Architecture	Procedure-oriented; knowledge is embedded in code	Separates Knowledge Base + Inference Engine + UI + Explanat
Reasoning	Built-in fixed algorithm	Inference engine (forward / backward chaining)
Explanation	Usually no explanation	Provides "why" and "how" explanations of conclusions
Modification	Hard — change code	Easy — just edit the rule base, no code change
Example	Chess-playing program, OCR	MYCIN (infections), DENDRAL (chemistry), CADUCEUS

Significance of the differentiating component: The key differentiator is the **separation of Knowledge Base from Inference Engine**. This separation gives expert systems three critical advantages:

- **Maintainability:** Domain rules can be updated without rewriting algorithms.
- **Explainability:** The system can trace which rules fired and justify decisions — vital in medicine, law, finance.
- **Transferability:** The same inference engine (shell) can serve many domains by swapping the KB.

Q2 A — Solve the following (10 marks)

Q2A i) Properties of task environment — Garden watering system. (5)

State each property and a one-line justification for the garden-watering agent:

Property	Value for Garden Watering	Justification (one line)
Fully / Partially Observable	Partially Observable	Sensors give only moisture/rain values — agent cannot see entire garden
Single / Multi-Agent	Single Agent	No other agent competes or cooperates; the watering system acts alone.
Deterministic / Stochastic	Stochastic	Outcome of watering depends on uncertain factors (evaporation, rain, leak
Episodic / Sequential	Sequential	Current watering affects future soil state — actions are not independent.
Static / Dynamic	Dynamic	Environment changes while the agent thinks (weather shifts, soil dries con
Discrete / Continuous	Continuous	Moisture, time, temperature are real-valued continuous quantities.
Known / Unknown	Known	Watering physics and effect on soil are well understood by designer.

Q2A ii) Problems in hill climbing process. State solutions. (5)

Hill Climbing is a local search algorithm that always moves to a neighbour with a better heuristic value. Three classic problems and their solutions:

1. Local Maximum: A peak that is higher than its neighbours but lower than the global maximum. The algorithm gets stuck because every neighbour looks worse.

- **Solution: Random-Restart Hill Climbing** — restart from different random initial states; **Simulated Annealing** — occasionally accept worse moves with decreasing probability.

2. Plateau (Flat region): A flat area of the search space where all neighbours have the same heuristic value. The algorithm cannot decide where to move.

- **Solution:** Take a **big jump** — move several steps in some direction; allow **sideways moves** (limited number) to traverse the plateau.

3. Ridge: A sequence of local maxima where the path to the top is not aligned with any single move direction; movement along principal axes leads downward.

- **Solution:** Use **multi-dimensional / diagonal moves**; or use **Stochastic Hill Climbing** which picks a random uphill move.

Additional advanced solutions: **Tabu search** (remember recently visited states), **Beam search** (track k best neighbours), and **Genetic Algorithms** for global exploration.

Q2 A (OR) — Agent architecture for garden watering system. (10)

The most suitable architecture is a **Model-based Reflex Agent with Goal-based extension** (or simply Model-based Reflex Agent if a strict on/off rule is enough). Justification: the environment is partially observable (single sensor doesn't capture full garden state) and dynamic; the agent must maintain internal state (recent watering, last rainfall) to avoid over-watering.

Components of a Model-based Reflex Agent (with sample contents for garden watering):

Block	Function	Sample Contents
Sensors	Perceive current environment	Soil moisture sensor reading = 18% ; Rain sensor = No ; Tank level = 70% ; Tim
State (internal memory)	Keep track of "what world is like now"	last_watered = 05:00 today ; total_water_today = 4 L ; last_rain = 2 days ago
Transition Model ("How world evolves")	Rules describing how state changes	If pump ON for 1 min → moisture rises ~5% ; If sun + no water → moisture drop
Sensor Model ("What my actions do")	How sensor reading reflects true state	Reading × 1.1 if temp > 35°C (correction factor)
Condition–Action Rules	If-then rules to choose action	IF moisture < 25% AND no rain forecast THEN turn pump ON IF moisture > 60% THEN turn pump OFF IF tank < 10% THEN raise alert
Actuators	Execute chosen action	Solenoid valve OPEN/CLOSE ; Pump START/STOP ; LED status

Block diagram (sketch in exam):

```
┌──────────────┐  Percepts  ┌──────────────────────────┐
│ Sensors │ ──────────────────▶│ State (current world) │
└──────────────┘  │ ▲ │
│ │ updated by model │
│ Transition + Sensor mdl │
│ │ │
│ ▼ │
│ Condition-Action Rules │
└──────────────┬──────────────────────────┘
│ action
▼

┌──────────┐
│ Actuators│ → Pump/Valve
└──────────┘
```

Q2 B — Solve any ONE (10 marks)

Q2B i) A* algorithm on given graph. (10)

Given edges and costs (from figure): S–A=1, A–B=2, A–D=1, B–D=5, A–C=1, C–D=3, D–G=4, S–G=10. **Heuristics:** h(S)=5, h(A)=3, h(B)=4, h(C)=2, h(D)=6, h(G)=0. **Start = S, Goal = G.**

A* uses f(n) = g(n) + h(n), where g = actual cost so far, h = estimated cost to goal. Always expand node with smallest f.

Step-by-step exploration (priority queue / OPEN list):

Step	Expand	Successors with g, h, f = g+h	OPEN after step
1	S (g=0,h=5,f=5)	A: g=1,h=3,f=4 ; G: g=10,h=0,f=10	{A(4), G(10)}
2	A (f=4)	B: g=1+2=3,h=4,f=7 ; C: g=1+1=2,h=2,f=4 ; D: g=1+1=2,h=6,f=8	{C(4), B(7), D(8), G(10)}
3	C (f=4)	D via C: g=2+3=5,h=6,f=11 — worse than existing D(8), skip	{B(7), D(8), G(10)}
4	B (f=7)	D via B: g=3+5=8,h=6,f=14 — worse, skip	{D(8), G(10)}
5	D (f=8)	G via D: g=2+4=6,h=0,f=6 — better than current G(10)! Update G to f=6, parent = D	{G(6)}
6	G (f=6)	GOAL reached. Stop.	—

(a) State-space tree (path explored): S → A → D → G (final accepted path after backtracking via parent pointers).

(b) Heuristic at each node on path: h(S)=5, h(A)=3, h(D)=6, h(G)=0.

(c) Solution path: S → A → D → G

(d) Total path cost: S→A (1) + A→D (1) + D→G (4) = **6**

Q2B ii) 0/1 Knapsack using Genetic Algorithm — one generation. (10)

Items i=1..7 with Profit p=[6,5,8,9,6,7,3] and Weight w=[2,3,6,7,5,9,4]; Capacity W=9.

Chromosome representation: Binary string of length 7. Bit i = 1 if item i is selected. e.g. 1010100 means items 1, 3, 5 are chosen.

Fitness function:

```
fitness(c) = Σ p_i × c_i if Σ w_i × c_i ≤ 9
fitness(c) = 0 otherwise (penalise overweight)
```

Step 1 — Initial Population (4 random chromosomes):

#	Chromosome (b1..b7)	Items chosen	Total weight	Total profit	Fitness
P1	1 0 1 0 0 0 0	1, 3	2+6 = 8 ≤ 9	6+8 = 14	14
P2	0 1 0 0 1 0 1	2, 5, 7	3+5+4 = 12 > 9	—	0 (infeasible)
P3	1 1 0 0 0 0 1	1, 2, 7	2+3+4 = 9 ≤ 9	6+5+3 = 14	14
P4	0 0 0 1 0 0 0	4	7 ≤ 9	9	9

Step 2 — Selection (Roulette wheel / tournament): Choose parents with probability ∝ fitness. P1, P3 (each 14) and P4 (9) are most likely; P2 (0) is rejected. Selected parents = **P1 and P3**.

Step 3 — Crossover (single-point at position 3):

```
Parent 1: 1 0 1 | 0 0 0 0
Parent 2: 1 1 0 | 0 0 0 1
Child 1 : 1 0 1 | 0 0 0 1 (items 1,3,7)
Child 2 : 1 1 0 | 0 0 0 0 (items 1,2)
```

Step 4 — Mutation (flip 1 random bit with low probability ~0.05): Suppose bit 5 of Child 1 flips:

```
Child 1 (mutated): 1 0 1 0 1 0 1 (items 1,3,5,7)
Weight = 2+6+5+4 = 17 > 9 → infeasible, fitness = 0
```

Step 5 — Evaluate new generation:

Chrom	Items	Weight	Profit	Fitness
Child 1 mut: 1 0 1 0 1 0 1	1,3,5,7	17	—	0
Child 2: 1 1 0 0 0 0 0	1,2	5	11	11
Survivor from old (elitism)	1,3 (P1)	8	14	14
Survivor from old (elitism)	1,2,7 (P3)	9	14	14

Best of generation 1: Profit = 14 with chromosome 1010000 (items 1, 3) — weight 8 — or 1100001 (items 1,2,7) — weight 9. Optimal final answer (after multiple generations) = items 1,2,4 with profit 20, weight 12 (infeasible) or items 1,2,5 (weight 10) etc.; ideal feasible optimum = items 1,3 → profit 14 OR items 1,2,4 (weight 12 ✗). Best feasible = profit 14.

Q3 — Solve any TWO (20 marks)

Q3 i) FOL conversion + Backward Chaining: Steve likes BJ301. (4+6)

Step 1 — Convert each English sentence into First-Order Logic:

- 1. Steve only likes easy courses: ∀x. Likes(Steve, x) → Easy(x)
- 2. Science courses are hard: ∀x. Science(x) → Hard(x)
- 3. All basket weaving courses are easy: ∀x. BasketWeaving(x) → Easy(x)
- 4. BJ301 is a basket weaving course: BasketWeaving(BJ301)

Implicit: **Hard ⇒ ¬Easy** (one cannot be both hard and easy) → ∀x. Hard(x) → ¬Easy(x).

Goal: *Likes(Steve, BJ301)*

Step 2 — Backward Chaining proof:

To prove Likes(Steve, BJ301) we look for a rule with Likes(Steve, x) on the consequent side. Rule 1 says Likes(Steve, x) → Easy(x) — this is in the wrong direction (it requires liking implies easy). However, if Steve *only* likes easy courses, then to prove he likes BJ301 we need BJ301 to be easy (this is consistent direction). We can take the contrapositive: **If BJ301 is easy AND Steve likes easy courses (general assumption), then Steve likes BJ301**. Strictly, with the given rule we cannot derive that he likes it. The course typically expects us to use the assumption that Steve likes ALL easy courses, i.e. ∀x. Easy(x) → Likes(Steve, x).

Re-interpreted Rule 1 (standard textbook reading): ∀x. Easy(x) → Likes(Steve, x)

Now backward chaining:

- Goal: Likes(Steve, BJ301)
- Sub-goal via Rule 1: prove Easy(BJ301)
- Sub-goal via Rule 3: prove BasketWeaving(BJ301)
- Fact 4 ⇒ BasketWeaving(BJ301) ✓ → therefore Easy(BJ301) ✓
- Therefore Likes(Steve, BJ301) ✓ — PROVED.

Inference rules used: *Universal Instantiation* (specialise ∀x to x = BJ301) and *Modus Ponens* (from P → Q and P infer Q).

Q3 ii) Convert sentences to FOL and then CNF. (10)

FOL:

- 1. Everyone has birthdays on some day → ∀x ∃d. HasBirthday(x, d)
- 2. Good dogs are always with their masters → ∀x. (Dog(x) ∧ Good(x)) → ∃y. (Master(y, x) ∧ With(x, y))
- 3. A barber shaves all men in the city who do not shave themselves → ∀x. (Man(x) ∧ InCity(x) ∧ ¬Shaves(x, x)) → Shaves(Barber, x)
- 4. Fred is a trained dog → Dog(Fred) ∧ Trained(Fred)
- 5. Every boy or girl is a child → ∀x. (Boy(x) ∨ Girl(x)) → Child(x)

CNF conversion (Skolemise existentials, drop universals, distribute):

- 1. CNF: HasBirthday(x, f(x)) — f is a Skolem function giving x's birthday.
- 2. CNF: {¬Dog(x) ∨ ¬Good(x) ∨ Master(g(x), x)}, {¬Dog(x) ∨ ¬Good(x) ∨ With(x, g(x))}
- 3. CNF: ¬Man(x) ∨ ¬InCity(x) ∨ Shaves(x, x) ∨ Shaves(Barber, x)
- 4. CNF: Dog(Fred), Trained(Fred)
- 5. CNF: ¬Boy(x) ∨ Child(x), ¬Girl(x) ∨ Child(x)

Q3 iii) Parcel delivery agent — uncertainties, plans, causes. (3+3+4)

(a) Key uncertainties in delivery process:

- **Traffic & weather:** Congestion, rain, accidents can delay arrival.
- **Customer availability:** Customer may not be present at the address during the chosen window.
- **Address accuracy:** Incorrect/incomplete pin code, lift broken, gate closed.

- **Vehicle / device failure:** Bike breakdown, GPS error, battery dead.
- **Package damage / loss:** Theft, fragile item, mis-loading at warehouse.
- **Cash-on-delivery refusal:** Customer refuses, payment problem.

(b) Two alternative plans (under uncertainty):

Plan A — Direct on-time delivery:

- Pickup parcel → Optimal route via GPS → Notify customer on departure → Deliver within slot → Collect OTP → Mark complete.
- Assumes traffic normal, customer available.

Plan B — Fallback plan with contingencies:

- If traffic > 30 min delay → Pre-call the customer, re-schedule or re-route through alternative road.
- If customer unavailable on arrival → Wait 5 minutes, then try contact, else drop at nearest pickup point / neighbour with consent.
- If vehicle breakdown → Hand-off to another agent in the area; update tracking.
- If COD refusal → Bring parcel back, mark RTO, schedule next attempt.

(c) Major causes of uncertainty:

- **Partial observability:** Agent does not know real-time customer location, true traffic ahead.
- **Non-determinism:** Same action may give different outcomes (e.g. ring the bell, may or may not be answered).
- **Imprecise sensors:** GPS error, inaccurate ETA models.
- **Dynamic environment:** Traffic, weather change over time.
- **Inaccurate / incomplete data:** Wrong address, outdated phone number.
- **Adversarial agents:** Theft, fraud, bad-faith claims.

Assumption: single delivery agent, single vehicle, GPS available, customer reachable by phone.

Q4 — Solve any TWO (20 marks)

Q4 ii) Bayesian Network — Burglary / Earthquake / Alarm / John / Mary calls. (2+4+2+2)

(a) Bayesian Network:

```
Burglary (B) Earthquake (E)
\ /
\ /
▼ ▼
Alarm (A)
/ \
▼ ▼
JohnCalls (J) MaryCalls (M)
```

Nodes: B, E (independent root causes); A depends on B and E; J and M each depend on A only.

(b) Probability tables (assumed):

P(B) = 0.001, P(E) = 0.002

B	E	P(A=T|B,E)
T	T	0.95
T	F	0.94
F	T	0.29
F	F	0.001

John (confuses doorbell): P(J=T|A=T)=0.90, P(J=T|A=F)=0.05

Mary (misses due to music): P(M=T|A=T)=0.70, P(M=T|A=F)=0.01

(c) P(A ∧ ¬B ∧ ¬E ∧ J ∧ M):

```
= P(¬B) × P(¬E) × P(A | ¬B, ¬E) × P(J | A) × P(M | A)
= 0.999 × 0.998 × 0.001 × 0.90 × 0.70
= 0.000628 ≈ 6.28 × 10⁻⁴
```

(d) P(¬A ∧ ¬B ∧ E ∧ ¬J ∧ ¬M):

```
= P(¬B) × P(E) × P(¬A | ¬B, E) × P(¬J | ¬A) × P(¬M | ¬A)
= 0.999 × 0.002 × (1 - 0.29) × (1 - 0.05) × (1 - 0.01)
= 0.999 × 0.002 × 0.71 × 0.95 × 0.99
= 0.001334 ≈ 1.33 × 10⁻³
```

Q4 ii) Expert System architecture — neat sketch, features, roles. (10)

An Expert System (ES) is a knowledge-based AI program that imitates the reasoning of a human expert in a narrow domain.

Architecture (sketch):

```
┌──────────┐  ┌──────────────┐  ┌──────────────┐
│ User │◀▶│ User Interface │◀──▶│ Explanation │
└──────────┘  └──────┬───────┘  │ Module │
│  └──────────────┘
▼ ▲
┌──────────────┐  │
│ Inference Engine │──────────┘
└──────┬───────┘
│
▼
```

```
┌──────────────────┐  ┌────────────────┐
│ Knowledge Base │◄►│ Knowledge Acq. │◄── Expert
│ (Rules + Facts) │  │ Module │
└──────────────────┘  └────────────────┘
```

Components and roles:

- **Knowledge Base (KB):** Stores domain rules (IF-THEN), facts, and heuristics from human experts. Example (MYCIN): IF organism is gram-negative AND morphology is rod AND patient is immunocompromised THEN suggest there is 0.6 evidence of Pseudomonas.
- **Inference Engine:** The reasoning component. Applies forward chaining (data-driven) or backward chaining (goal-driven) to derive conclusions from KB.
- **User Interface:** Allows the end-user to ask queries and receive recommendations in natural language.
- **Explanation Module:** Answers 'Why this question?' and 'How was this conclusion reached?' — critical for trust.
- **Knowledge Acquisition Module:** Tool for the knowledge engineer / expert to add or update rules without programmer help.
- **Working Memory:** Holds the current case data — symptoms, facts being asserted during a session.

Characteristic features: high domain performance ; rule-based ; explanation capability ; uncertainty handling (certainty factors / Bayesian) ; transparent ; easy to update.

Example — MYCIN: diagnoses bacterial infections of the blood and recommends antibiotics with ~70% expert accuracy. Other examples: DENDRAL (chemical structure), PROSPECTOR (mineral exploration), XCON (computer configuration at DEC).

Q4 iii) Action Description Language (ADL) — drone parcel exchange. (2+8)

ADL (Action Description Language): A planning language that extends STRIPS. It allows: negative pre-conditions, quantified preconditions/effects, conditional effects, and equality. Each action is described with parameters, preconditions, and effects (Add and Delete lists implicit through the effect formula).

Domain: Two drones D1 and D2 stationed at distinct locations L1 and L2 exchange packages P1 (currently at L1, target L2) and P2 (at L2, target L1).

Initial State:

```
At(D1, L1) ∧ At(D2, L2)
At(P1, L1) ∧ At(P2, L2)
Free(D1) ∧ Free(D2)
```

Goal State:

```
At(P1, L2) ∧ At(P2, L1)
```

Action 1 — Pickup(d, p, l):

```
Precondition: At(d, l) ∧ At(p, l) ∧ Free(d)
Effect: Carries(d, p) ∧ ¬At(p, l) ∧ ¬Free(d)
```

Action 2 — Fly(d, from, to):

```
Precondition: At(d, from) ∧ from ≠ to
Effect: At(d, to) ∧ ¬At(d, from)
Conditional effect: ∀p. Carries(d,p) → At(p,to) ∧ ¬At(p,from)
```

Action 3 — Drop(d, p, l):

```
Precondition: At(d, l) ∧ Carries(d, p)
Effect: At(p, l) ∧ ¬Carries(d, p) ∧ Free(d)
```

Sample plan:

- 1. Pickup(D1, P1, L1)
- 2. Fly(D1, L1, L2)
- 3. Drop(D1, P1, L2) → goal partial: At(P1, L2) ✓
- 4. Pickup(D2, P2, L2)
- 5. Fly(D2, L2, L1)
- 6. Drop(D2, P2, L1) → goal: At(P2, L1) ✓

(Steps 1-3 and 4-6 can run in parallel since drones are independent — ADL handles this naturally.)

Q5 — Solve any FOUR (20 marks)

Q5 i) Planning vs Searching. (5)

Aspect	Searching	Planning
State Representation	Atomic / opaque states	Structured states (sets of facts / fluents)
Actions	Successor function — produces next state	Each action has preconditions, add-list, delete-list
Decomposition	Cannot decompose problem	Can decompose into sub-problems (goal regression)
Heuristics	Domain-specific, hand-crafted	Domain-independent heuristics derivable from action descript
Branching	High, sees all actions	Considers only actions relevant to current sub-goal
Example	BFS finding path in maze	Block-world: rearrange blocks A,B,C using Stack/Unstack acti

In short — **searching is generic blind/informed traversal**; **planning uses structured representations and decomposition** so it scales to far larger real-world tasks (logistics, robotics, factory scheduling).

Q5 ii) Hierarchical Task Planning — Constructing a villa. (5)

Hierarchical Task Network (HTN) planning decomposes a high-level complex task into a hierarchy of smaller, more concrete sub-tasks, until each leaf is a primitive (directly executable) action.

Example: Build a villa after buying a plain land — decomposition tree:

```
BuildVilla
├── AcquireLand
│   ├── SurveyLand
│   ├── NegotiatePrice
│   └── RegisterAtSubRegistrar
├── ObtainApprovals
│   ├── GetBuildingPlanSanctioned
│   └── GetEnvironmentalNoC
├── ConstructVilla
│   ├── DigFoundation
│   ├── BuildStructure (RCC columns + slabs)
│   ├── BuildWalls (brickwork + plaster)
│   ├── InstallUtilities (electrical, plumbing, HVAC)
│   └── DoFinishing (paint, flooring, fixtures)
└── HandoverAndOccupy
    ├── GetOccupancyCertificate
    └── MoveIn
```

Why HTN is useful: Each node may have ordering constraints (foundation before walls), can be re-planned independently if a sub-goal fails (e.g. plumbing redo), and large real-world projects become tractable.

Q5 iii) Four approaches of AI. (5)

AI is studied along two dimensions — Thought vs Behaviour, Human-like vs Rational — giving four approaches:

Approach	Definition	Key technique / example
1. Thinking Humanly	Make computers think like humans — model cognitive processes	Cognitive science, Newell & Simon GPS, neural models
2. Acting Humanly	Make computers behave indistinguishably from humans	Turing Test (1950) — NLP, knowledge representation, autom
3. Thinking Rationally	Use logic; computers should reason correctly	Aristotle syllogisms, logicist AI, Prolog, theorem provers
4. Acting Rationally	Build agents that act to maximise expected outcome	Rational agents / utility theory — modern AI (RL, planning, M

Modern AI predominantly follows the **Acting Rationally / Rational Agent** approach — it is more general (covers both reasoning and reflexive action), and mathematically tractable.

Q5 iv) Short note on Natural Language Processing. (5)

Natural Language Processing (NLP) is the branch of AI that enables computers to understand, interpret, generate, and respond to human (natural) languages such as English, Hindi or Marathi.

Why hard? Natural language has ambiguity (lexical, syntactic, semantic), context-dependence, anaphora, idioms, sarcasm, code-mixing, evolving slang.

Levels / phases of NLP:

- **Lexical analysis / tokenisation:** Split text into words/tokens; stemming, lemmatisation.
- **Syntactic analysis (parsing):** Build grammatical structure (parse tree); POS tagging.
- **Semantic analysis:** Map words / sentences to meaning; word-sense disambiguation; named-entity recognition.
- **Discourse integration:** Resolve pronouns and coreference across sentences.
- **Pragmatic analysis:** Use real-world context, speaker intent, sarcasm.

Applications: machine translation (Google Translate), chatbots (ChatGPT), sentiment analysis, voice assistants (Alexa, Siri), spam filters, summarisation, question-answering, information retrieval, OCR.

Q5 v) Knowledge-based agent architecture — Wumpus World example. (5)

A **Knowledge-Based (KB) Agent** maintains an internal Knowledge Base (a set of sentences in some formal language) and uses inference to decide what action to take.

Generic skeleton:

```
function KB-AGENT(percept) returns action
static: KB, t (counter)
TELL(KB, MakePerceptSentence(percept, t))
action ← ASK(KB, MakeActionQuery(t))
TELL(KB, MakeActionSentence(action, t))
t ← t + 1
return action
```

Wumpus World example: 4×4 grid; cells may contain pits, a Wumpus, or gold. The agent perceives [Stench, Breeze, Glitter, Bump, Scream].

- **Sensors → percepts:** Breeze means adjacent pit; Stench means adjacent Wumpus; Glitter means gold in current square.
- **TELL KB:** 'At [1,1] no Breeze and no Stench' → infer [1,2] and [2,1] are safe.
- **ASK KB:** 'Is [1,2] safe?' → If yes, move there.
- **Inference:** Rules like ¬Breeze(x,y) → ¬Pit(adjacent(x,y)) let the agent prove safe squares using resolution / forward chaining.
- **Actuators:** Forward, TurnLeft, TurnRight, Grab, Shoot, Climb.

The Wumpus World illustrates how a logical KB-agent reasons under partial observability and acts safely.

Q5 vi) Forms of Learning. (5)

- **1. Supervised Learning:** Learns a function from labelled training examples (input → known output). Examples: classification (spam vs ham), regression (house price). Algorithms: Decision Trees, SVM, Neural Networks, Linear Regression.
- **2. Unsupervised Learning:** Learns hidden structure from unlabelled data. Examples: customer segmentation (K-means), dimensionality reduction (PCA), association (market-basket).
- **3. Semi-supervised Learning:** Small amount of labelled + large unlabelled data. Common in medical imaging where labels are expensive.
- **4. Reinforcement Learning (RL):** Agent learns from reward/punishment by interacting with an environment. Used in robotics, game-playing (AlphaGo), self-driving. Algorithms: Q-learning, SARSA, DQN, PPO.
- **5. Online / Incremental Learning:** Learns continuously as data arrives one example at a time.
- **6. Transfer Learning:** Reuses knowledge learnt on one task for a related task (e.g. ImageNet → medical X-rays).
- **7. Self-supervised Learning:** Generates its own labels from unlabelled data (e.g. predicting next word in BERT/GPT).

PAPER 2 : JAN 2024 – APR 2024

Q1 — Solve any FOUR (20 marks)

Q1 i) Basic components of Intelligent systems. (5)

An Intelligent System receives input from the world, reasons, and acts upon it. Its basic components are:

- **Perception (Sensors / Input unit):** Captures data from environment — cameras, microphones, sensors, keyboard.
- **Knowledge Representation:** Internal model of the world — facts, rules, ontologies, semantic networks.
- **Reasoning / Inference engine:** Draws conclusions from knowledge — forward/backward chaining, search, probabilistic inference.
- **Learning component:** Acquires new knowledge from experience or data — ML algorithms, neural networks.
- **Planning component:** Generates a sequence of actions to achieve goals.
- **Natural Language Processing:** Communicates with humans in spoken/written language.
- **Actuators / Action unit:** Executes decisions in the real world — motors, wheels, displays, network commands.

Q1 ii) Limitations of 'Thinking Rationally' approach. (5)

The **Thinking Rationally** approach (laws of thought, logicist AI) tries to make machines reason using formal logic — e.g., Aristotle's syllogisms, predicate logic, Prolog.

Limitations:

- **1. Informal knowledge:** Most real-world knowledge cannot be expressed in 100% precise logical form (e.g. "this looks like a chair").
- **2. Computational intractability:** Even small problems can lead to combinatorial explosion when solving them with logical inference.
- **3. Uncertainty:** Classical logic is binary; real life requires probability, fuzziness, defaults (e.g. 'birds usually fly').
- **4. Difference between proving and acting:** The approach focuses on producing correct conclusions, but not on what to do under time-pressure / partial observability.
- **5. Knowledge acquisition bottleneck:** Hand-coding every rule is infeasible.
- **6. No learning:** Pure logic systems don't naturally improve with data — modern AI prefers learning + rational action over pure deduction.

Q1 iii) PEAS for automated driving. (5)

Component	Description for Automated (Self) Driving Car
Performance	• Safety (no accidents, no traffic violations) • Reach destination on time • Passenger comfort (smooth acceleration / braking) • Fuel / battery efficiency • Legal compliance (speed limits, signals)
Environment	• Roads (highways, city streets, parking) • Other vehicles, pedestrians, animals, cyclists • Traffic signals, signs, lane markings • Weather (rain, fog, night) • Construction zones, potholes
Actuators	• Steering wheel, accelerator, brake • Gear / transmission • Indicators, horn, headlights, wipers • Display & voice to passengers

Sensors	• Cameras (front, side, rear) • LiDAR, RADAR, ultrasonic sensors • GPS / IMU • Speedometer, odometer • Tire pressure, engine status

Task environment for self-driving: Partially observable, multi-agent, stochastic, sequential, dynamic, continuous, known.

Q1 iv) Two applications of AI. (5)

1. Medical diagnosis: AI systems analyse X-rays, MRIs, CT scans, ECGs and lab reports to assist doctors. Deep-learning models such as Google's DeepMind detect diabetic retinopathy with super-human accuracy; IBM Watson Oncology suggests cancer treatments; AI chatbots handle initial triage. Outcome: faster, more accurate diagnosis, especially in remote regions.

2. Self-driving cars / Autonomous Vehicles: Companies like Tesla, Waymo and Cruise use AI (deep CNNs + LiDAR + sensor fusion + reinforcement learning) to perceive surroundings, predict behaviour of other agents, plan trajectory, and control steering/throttle/brake. Outcome: reduced human-error accidents, mobility for elderly/disabled, transformation of logistics.

Other notable applications: NLP chatbots (ChatGPT), recommendation engines (Netflix, Amazon), fraud detection in banking, agricultural drones, weather forecasting, virtual personal assistants.

Q1 v) Advantage of Expert Systems. (5)

- **Expertise availability 24×7:** Captures rare human expert knowledge and makes it accessible anytime, anywhere.
- **Consistent decisions:** Same input always produces the same answer — no fatigue, no emotion, no bias from mood.
- **Cost effective:** Reduces dependence on expensive human experts after initial development.
- **Explanation capability:** Justifies decisions via 'why' / 'how' modules — builds user trust.
- **Handles complex domains:** Can apply hundreds of interacting rules faster than a human.
- **Preservation of knowledge:** Knowledge of retiring experts stays usable in the KB.
- **Training tool:** Acts as a tutor for juniors learning the domain.
- **Modular & maintainable:** Adding a rule does not require rewriting code.

Q1 vi) ADL — wearing socks and shoes. (5)

Initial state: ¬OnLeftSock ∧ ¬OnRightSock ∧ ¬OnLeftShoe ∧ ¬OnRightShoe

Goal: OnLeftShoe ∧ OnRightShoe

Actions in ADL:

```
Action: PutLeftSock
Precondition: ¬OnLeftSock
Effect: OnLeftSock

Action: PutRightSock
Precondition: ¬OnRightSock
Effect: OnRightSock

Action: PutLeftShoe
Precondition: OnLeftSock ∧ ¬OnLeftShoe
Effect: OnLeftShoe

Action: PutRightShoe
Precondition: OnRightSock ∧ ¬OnRightShoe
```

```
Effect: OnRightShoe
```

Valid plan: PutLeftSock → PutRightSock → PutLeftShoe → PutRightShoe. (Sock-actions and shoe-actions of opposite feet are independent, so partial-order planning would parallelise them.)

Q2 A — Solve the following (10 marks)

Q2A i) Bayesian Network — grass wet (sprinkler / rain). (5)

When to use Bayesian Networks:

- When a problem involves **multiple random variables with conditional dependencies**.
- When we need to reason under **uncertainty** (probabilities, not deterministic logic).
- To efficiently encode the **joint probability distribution** in a compact graphical form.
- Useful for **diagnosis** (medical, fault), **prediction**, and **causal reasoning**.

Topology:

```
Cloudy (C)
/ \
▼ ▼
Sprinkler(S) Rain(R)
\ /
▼ ▼
WetGrass (W)
```

Assumed Conditional Probability Tables:

P(C=T)	P(C=F)
0.5	0.5

P(S | C): C=T → 0.10 ; C=F → 0.50

P(R | C): C=T → 0.80 ; C=F → 0.20

S	R	P(W=T|S,R)
T	T	0.99
T	F	0.90
F	T	0.90
F	F	0.00

Required probabilities (computed by enumeration over C):

- (a) Sprinkler on only: P(W=T | S=T, R=F) = 0.90
- (b) Raining only: P(W=T | S=F, R=T) = 0.90
- (c) Both: P(W=T | S=T, R=T) = 0.99

Q2A ii) Compare A* vs AO* with example. (5)

Aspect	A* Search	AO* Search
Problem type	OR graphs — only one alternative at a time	AND-OR graphs — sub-problems must all be solved
Goal	Single goal state	May have several sub-goals to be solved together
Cost	f(n) = g(n) + h(n) for a single path	f(n) = sum of f-values of all AND children, plus h
Re-evaluation	Only forward updates needed	Backward propagation to re-evaluate ancestors when child cost ch
Optimality	Optimal if h is admissible	Optimal if h is admissible AND no loops
Use case	Path finding, route planning	Theorem proving, problem reduction (Tower of Hanoi)
Example	Find shortest path Mumbai→Delhi	Assemble car = AND(make body, make engine, OR(assemble mar

Q2 A (OR) — Crypt-arithmetic TWO + TWO = FOUR via CSP backtracking. (10)

Variables: T, W, O, F, U, R — each gets a distinct digit 0–9. T and F are leading digits → T ≠ 0, F ≠ 0.

Carries: C1, C2, C3 ∈ {0, 1}

Constraints (column by column from right):

```
Column 1 (units): O + O = R + 10·C1
Column 2 (tens): W + W + C1 = U + 10·C2
Column 3 (hund.): T + T + C2 = O + 10·C3
Column 4 (thou.): C3 = F
All-different(T,W,O,U,F,R) ; T,F ≠ 0
```

Heuristic chosen: *Most-Constrained Variable* (MRV) — assign first the variable that participates in the most constraints. Here F is forced quickly: from Col 4, F = C3 ∈ {0,1}, but F ≠ 0 → F = 1.

Step-by-step backtracking:

- F = 1 (from C3 = F and F ≠ 0).
- Hence C3 = 1 → 2T + C2 = O + 10. Thus 2T + C2 ≥ 10 → T ≥ 5.
- Try T = 7 → 14 + C2 = O + 10 → O = 4 + C2 ∈ {4, 5}.
- Take O = 4 → C2 = 0 → 2W + C1 = U (no carry). Col1: 2O = R + 10C1 → 8 = R + 10C1 → C1=0, R=8.
- Now 2W = U. Used digits so far: {1,4,7,8}. Try W = 6 → U = 12 ✗. Try W = 3 → U = 6. Used = {1,3,4,6,7,8} – all distinct. ✓

Final assignment: T = 7, W = 3, O = 4, F = 1, U = 6, R = 8.

Verification: 734 + 734 = 1468. ✓ (TWO + TWO = FOUR)

Q2 B — Solve any ONE (10 marks)

Q2B i) Alpha-Beta pruning over Min-Max. How α and β chosen. (10)

Min-Max problem: the algorithm explores the entire game tree — exponential in depth. Many branches give no new information about the root value.

Alpha-Beta pruning is an optimisation of Min-Max: while traversing the tree depth-first, we maintain two values:

- **α (alpha):** the best (highest) value the MAX player can guarantee so far along the path.
- **β (beta):** the best (lowest) value the MIN player can guarantee so far along the path.

Pruning condition: If at any node α ≥ β, stop exploring further siblings of that node — they cannot affect the final decision. This is called *β-cut* at MAX nodes and *α-cut* at MIN nodes.

How α and β are chosen:

- **Initialise** at the root: α = −∞, β = +∞.
- **Pass** α and β down to children unchanged when recursing.
- **At MAX node:** after evaluating each child v, α = max(α, v); if α ≥ β, prune remaining children.
- **At MIN node:** after evaluating each child v, β = min(β, v); if α ≥ β, prune remaining children.
- **Return** the v value to the parent.

Example (small tree): MAX → [MIN(3, 5, 6), MIN(6, 7, 4, 5)]

- Left MIN node: child=3 → β=3; child=5 → β=min(3,5)=3; child=6 → β=3. Value=3. α at root becomes max(−∞,3)=3.

- Right MIN node: child=6 → β=6; but α=3 < β=6, continue. child=7 → β=6; child=4 → β=4; child=5 → β=4. But once β ≤ α (here when β reaches 3 or less), prune. The remaining children of MIN can be skipped. Value=min ≥ 3 only matters.
- Root value = max(3, 4) = 4.

Best-case complexity: with perfect move ordering, alpha-beta reduces effective branching factor to √b; explores O(b^(d/2)) nodes — same time, double depth.

Q2B ii) Graph Coloring via Genetic Algorithm — one generation. (10)

Undirected graph with 5 vertices A, B, C, D, E. Edges shown: A–B, B–C, A–D, B–D, B-E, D–E (assumed from the figure). Colours = 3 (numbered 1, 2, 3).

Chromosome: length-5 string where position i = colour of vertex i. e.g. (1, 2, 1, 2, 1) means A=1, B=2, C=1, D=2, E=1.

Fitness function:

```
conflicts(c) = # of edges (u, v) such that colour(u) = colour(v)
fitness(c) = 1 / (1 + conflicts(c)) — higher is better
```

Step 1 — Initial population (4 random chromosomes):

#	Chromosome (A B C D E)	Conflicts (attacking pairs)	Fitness
P1	1 2 1 2 1	AB(1,2)ok; BC(2,1)ok; AD(1,2)ok; BD(2,2)✗; BE(2,1)ok; DE(2,1)ok → 1	1/2 = 0.50
P2	1 2 3 2 3	AB ok; BC ok; AD ok; BD(2,2)✗; BE ok; DE(2,3)ok → 1	1/2 = 0.50
P3	2 1 2 3 1	AB ok; BC ok; AD ok; BD(1,3)ok; BE(1,1)✗; DE(3,1)ok → 1	1/2 = 0.50
P4	1 2 1 3 2	AB ok; BC ok; AD ok; BD ok; BE(2,2)✗; DE(3,2)ok → 1	1/2 = 0.50

Step 2 — Selection: All have same fitness 0.5 — pick any two by tournament. Say P1 and P4 are selected as parents.

Step 3 — Crossover (single-point between positions 3 and 4):

```
Parent P1: 1 2 1 | 2 1
Parent P4: 1 2 1 | 3 2
Child C1: 1 2 1 | 3 2 (= P4 here)
Child C2: 1 2 1 | 2 1 (= P1 here)
```

Step 4 — Mutation (flip a random gene with prob ~0.1): Suppose gene 4 of C1 mutates from 3 → 1:

```
C1 mutated: 1 2 1 1 2
Conflicts: AD(1,1)✗; BD(2,1)ok; BE(2,2)✗; DE(1,2)ok → 2 conflicts; fitness = 1/3 ≈ 0.33
```

Step 5 — Evaluate new generation: Best chromosome so far still P3 (or equivalent with 1 conflict), fitness 0.5. After multiple generations, GA converges to a 0-conflict solution such as **(1, 2, 3, 1, 3)** — A=1, B=2, C=3, D=1, E=3 — verify: AB(1,2)✓ BC(2,3)✓ AD(1,1)✗ — actually wrong. Correct solution example: **(1, 2, 1, 3, 1)** if BE-edge absent, else **(1, 3, 1, 2, 3)**. After exploring further generations, GA finds a feasible 3-colouring; this iteration shows the mechanics.

Q3 — Solve any TWO (20 marks)

Q3 i) Intelligent Agents. Simple reflex agent for mobile predictive text. (10)

Intelligent Agent: Anything that perceives its environment through sensors and acts upon that environment through actuators to achieve its goals. Formally, agent = function f : Percept Sequence → Action.

Types of agents (in increasing sophistication):

- **Simple reflex agent** — acts only on current percept using condition-action rules.
- **Model-based reflex agent** — maintains internal state of the world.
- **Goal-based agent** — considers what action achieves a goal.
- **Utility-based agent** — considers a utility/quality measure over outcomes.
- **Learning agent** — improves performance with experience.

Simple reflex agent for predictive text on mobile keyboard:

Predictive text suggests the next word as the user types. A simple reflex agent maps the last typed prefix (percept) to a most-likely word (action) using a precomputed lookup table.

```
Sensor: characters entered so far in the current word + previous word
Action: show top-3 next-word predictions
Condition-Action rules (examples):
IF prefix = 'th' THEN suggest 'the' / 'this' / 'that'
IF previous_word = 'good' AND prefix = 'mor' THEN suggest 'morning'
IF prefix = 'happy birth' THEN suggest 'birthday'
```

Why it qualifies as **simple reflex**: It does not store user history or learn — it just matches the current percept to a rule and outputs an action. (Real Gboard / SwiftKey go beyond this — they are learning agents that personalise via user typing history.)

Q3 ii) FOL + Forward Chaining — Gita likes almond. (10)

FOL representation:

- 1. Gita likes all kinds of food → ∀x. Food(x) → Likes(Gita, x)
- 2. Mango and Chapati are food → Food(Mango) ∧ Food(Chapati)
- 3. Gita eats almond and is still alive → Eats(Gita, Almond) ∧ Alive(Gita)
- 4. Anything anyone eats and isn't killed by, is food → ∀x ∀y. (Eats(y, x) ∧ ¬Killed(y, x)) → Food(x)
- Goal: Likes(Gita, Almond)

Implicit assumption: If a person is alive after eating something then they are not killed by it, i.e. ∀x ∀y. Alive(y) ∧ Eats(y,x) → ¬Killed(y,x). [or take ¬Killed(Gita, Almond) as a derivable fact since Alive(Gita) is asserted].

Forward chaining steps:

- Facts known: Eats(Gita, Almond), Alive(Gita), Food(Mango), Food(Chapati).
- Step 1: From Alive(Gita) and Eats(Gita, Almond) infer ¬Killed(Gita, Almond) (by the alive-not-killed assumption). [Inference rule: **Modus Ponens**]
- Step 2: Apply rule 4 with y = Gita, x = Almond: from Eats(Gita, Almond) ∧ ¬Killed(Gita, Almond) infer Food(Almond). [**Universal Instantiation** + **Modus Ponens**]
- Step 3: Apply rule 1 with x = Almond: from Food(Almond) infer Likes(Gita, Almond). [**Universal Instantiation** + **Modus Ponens**]
- Goal Likes(Gita, Almond) is now in the KB → **PROVED**.

Inference rules used: Universal Instantiation, Modus Ponens, And-Elimination.

Q3 iii) Typical steps for Natural Language Processing. (10)

Text → Output goes through the following pipeline (overlap is common; modern transformer models compress many steps end-to-end).

- **1. Text Acquisition:** Collect raw text from documents, web, speech-to-text.
- **2. Pre-processing:** Lower-casing, removing punctuation/HTML, expanding contractions, removing stop-words.
- **3. Tokenisation:** Split text into tokens (words, subwords or characters). e.g. 'I'm running' → ['I','am','running'].
- **4. Morphological / Lexical analysis:** Stemming (running → run), lemmatisation (better → good), spell-correction.
- **5. POS tagging:** Assign parts of speech (noun, verb, adj) to each token using HMM / CRF / Transformers.
- **6. Syntactic parsing:** Build parse-tree using context-free grammar — discovers grammatical structure.
- **7. Named-Entity Recognition (NER):** Identify persons, places, organisations, dates.
- **8. Semantic analysis:** Map words/sentences to meaning representations (logical forms, embeddings).
- **9. Discourse analysis / Coreference resolution:** Link pronouns to entities across sentences.
- **10. Pragmatic analysis:** Use real-world context, intent, speech-acts (request/command/question).
- **11. Task-specific output:** Translation / summarisation / sentiment / answer / response, etc.

Applications: machine translation, chatbots, sentiment analysis, search engines, voice assistants, summarisation, text-to-speech, OCR, plagiarism detection.

Q4 — Solve any TWO (20 marks)

Q4 i) Hill Climbing Search — algorithm, problems, solutions. (10)

Hill Climbing is a local search algorithm that continually moves in the direction of increasing heuristic value (uphill) and terminates when no neighbour is better.

Algorithm (Simple Hill Climbing):

```
function HillClimb(problem):
current ← MakeNode(problem.InitialState)
loop:
neighbour ← a successor of current
if value(neighbour) ≤ value(current): return current # local peak
current ← neighbour
```

Variants: Steepest-Ascent (pick best neighbour, not first), Stochastic Hill Climbing (random uphill), First-choice Hill Climbing (random neighbours until improvement), Random-Restart Hill Climbing.

Problems:

- **Local maximum** — higher than neighbours but lower than global; algorithm gets stuck.
- **Plateau** — flat region; no neighbour is better, algorithm cannot choose direction.
- **Ridge** — sequence of local maxima where movement along axes is downward, but a diagonal path leads up.

Solutions:

- **Random-Restart Hill Climbing** — run from many random starts; keep the best result. With enough restarts, eventually hits global maximum.
- **Simulated Annealing** — occasionally accepts worse moves with probability e^(−ΔE/T); temperature T decreases over time. Escapes local maxima.
- **Allow sideways moves** — limited number of equal-value moves to traverse plateaus.
- **Stochastic / first-choice Hill Climbing** — randomness helps explore.
- **Beam Search** — keep k best states instead of one to reduce risk.
- **Genetic Algorithms / Tabu Search** — population-based and memory-based exploration.

Q4 ii) Knowledge engineering for COVID-19 detection. (10)

Knowledge engineering is the process of building a knowledge base for an expert system. Standard 7-step process (Russell & Norvig):

Step	Task	For COVID-19 detection
1	Identify the task	Diagnose whether a patient is likely infected with COVID-19 given their symptoms / contact / test histo
2	Assemble relevant knowledge	WHO / ICMR guidelines, symptom list (fever, dry cough, loss of smell, breathlessness), comorbidities,
3	Decide on vocabulary (predicates, functions, constants)	HasSymptom(patient, symptom), TempCelsius(patient, t), Contact(patient, p2), Tested(patient, result),
4	Encode general domain knowledge	Rules e.g. R1: HasSymptom(P, Fever) ∧ HasSymptom(P, DryCough) ∧ HasSymptom(P, LossOfSmell) → Likely(R2: Tested(P, Positive) → Definite(P, COVID). R3: Contact(P, Q) ∧ Definite(Q, COVID) ∧ HasSymptom(P, Fever) → Suspected(P, COVID).
5	Encode the specific problem instance	For patient X: HasSymptom(X, Fever), HasSymptom(X, DryCough), Contact(X, Y), Tested(Y, Positive)
6	Pose queries to the inference engine	ASK: Likely(X, COVID)? — engine uses backward chaining or resolution and returns Yes/No with expl
7	Debug the knowledge base	Validate against real cases, add missing rules (e.g. asymptomatic carriers, new variants), tune certaint

Q4 iii) Expert system for automated driving in mountains. (10)

Architecture (standard 5-component diagram): User Interface ↔ Inference Engine ↔ Knowledge Base; Knowledge Acquisition module fed by domain experts; Explanation module.

Sample contents in each component:

- **Knowledge Base:** rules learnt from experienced mountain drivers — e.g. R1: IF gradient > 8% AND wet road THEN use lower gear AND reduce speed by 30%. R2: IF sharp bend ahead AND visibility < 50m THEN honk AND keep left. R3: IF altitude > 3000m THEN monitor engine air-fuel mixture.
- **Inference Engine:** forward chaining over current sensor readings; backward chaining for diagnostics ("why did the system brake?").
- **Working Memory:** current sensor data — speed, gradient, road type, visibility, weather.
- **User Interface:** dashboard + voice — alerts driver, accepts overrides.
- **Explanation Module:** "Why slower?" → "Because gradient 12%, wet, and a hair-pin bend in 80m."
- **Knowledge Acquisition:** Used by domain experts (rally drivers) to add rules without coding.

Three inference rules:

- R1: IF gradient > 8% AND road = wet THEN gear ← Low AND speed_limit ← 0.7 × current.
- R2: IF visibility < 50 m AND road_type = curve THEN honk ← ON AND lane ← left.
- R3: IF temp_engine > 100°C AND altitude > 3000 m THEN pull_over_and_cool ← ON.

Q5 — Write notes on any FOUR (20 marks)

Q5 i) Properties of task environment. (5)

Russell & Norvig identify 7 key properties of a task environment:

- **Fully vs Partially Observable:** Whether sensors give complete state info. Chess = fully observable; Poker = partially.
- **Single vs Multi-agent:** Does the environment contain other agents whose actions matter? Crossword = single; football = multi-agent.
- **Deterministic vs Stochastic:** Is the next state fully determined by current state and action? Sudoku = deterministic; dice game = stochastic.
- **Episodic vs Sequential:** Are actions independent (episodic) or does current decision affect future? Image classification = episodic; chess = sequential.
- **Static vs Dynamic:** Does environment change while the agent thinks? Sudoku = static; driving = dynamic.
- **Discrete vs Continuous:** Are time, states, percepts discrete or continuous? Chess = discrete; driving = continuous.
- **Known vs Unknown:** Does the agent know the rules / outcomes of actions? Card-game with new rules = unknown.

Q5 ii) Lonavla rainy season trip — family of 10, ages 1–70. (5)

Plan A — Cottage Stay Plan (relaxed):

- Day 1: Drive Mumbai → Lonavla; check-in to a 4-bedroom cottage with indoor games. Lunch + monsoon-view tea.
- Day 2: Bhushi dam at early morning (less crowd, safer); Karla & Bhaja caves (covered, suitable for elders & toddlers); return by 6 PM.
- Day 3: Visit Pawna lake-side resort; light walking; head back to Mumbai by 5 PM.

Plan B — Day-trip Plan (single day, no overnight):

- Mumbai → Lonavla by 9 AM; visit Tiger Point + Lion's Point (viewpoint, safe for all).
- Lunch at a hilltop hotel with covered seating.
- Karla caves OR chikki / fudge shopping in the market.
- Return to Mumbai by 7 PM.

Uncertainties:

- Heavy rain / landslides closing the Expressway.
- Bhushi dam overflow / accidents (recurrent annual issue).
- Elder family members (age 70) — slippery walks, knee strain.
- Toddler (age 1) — sensitive to cold / damp, needs frequent feed.
- Vehicle issues (skidding, fog).
- Hotel/cottage availability (peak monsoon weekends overbooked).
- Traffic jams returning Sunday night.

Q5 iii) Differentiate Planning vs Searching. (5)

Already covered in Paper 1 Q5 i. Brief recap:

- **Searching** = generic state-space traversal; states atomic; actions opaque; e.g. BFS, A*, IDA*.
- **Planning** = uses structured state representation (sets of fluents) with explicit pre/post-conditions; allows decomposition and domain-independent heuristics; e.g. STRIPS, ADL, GraphPlan, POP.
- Planning scales to larger real-world problems (manufacturing, logistics) where pure search blows up combinatorially.

Q5 iv) Uninformed vs Informed Search. (5)

Feature	Uninformed (Blind)	Informed (Heuristic)

Information used	Only problem definition (graph + goal)	Additional domain heuristic h(n)
Goal directedness	Does not know direction of goal	Estimates how close each node is to goal
Efficiency	Often explores many irrelevant states	Far fewer node expansions on average
Optimality	BFS optimal if step costs equal; UCS optimal	A* optimal if h is admissible
Examples	BFS, DFS, Iterative Deepening, UCS	Best-first, Greedy, A*, AO*, IDA*

Q5 v) Universal and existential quantifier. (5)

Universal quantifier (∀): 'for all'. The formula ∀x. P(x) is true if P(x) holds for every element x in the domain. Example: ∀x. (Human(x) → Mortal(x)) — every human is mortal.

Existential quantifier (∃): 'there exists at least one'. The formula ∃x. P(x) is true if at least one x in the domain satisfies P(x). Example: ∃x. (Student(x) ∧ Topper(x)) — at least one student is a topper.

Important facts:

- ∀ usually pairs with → (implication); ∃ usually pairs with ∧ (conjunction).
- Negation duality: ¬∀x. P(x) ≡ ∃x. ¬P(x); ¬∃x. P(x) ≡ ∀x. ¬P(x).
- Order matters: ∀x ∃y. Loves(x, y) ≠ ∃y ∀x. Loves(x, y).
- Universal Instantiation: from ∀x. P(x) infer P(a) for any constant a.
- Existential Instantiation: from ∃x. P(x) infer P(c) where c is a new Skolem constant.

Q5 vi) Things AI cannot do in a technical forum. (5)

- **Genuine creativity:** Cannot invent a fundamentally new scientific theory; only re-combines patterns.
- **True common-sense reasoning:** Cannot reliably understand everyday physics (water flows downhill) and human contexts.
- **Emotions / empathy:** Cannot feel — only mimic emotional responses.
- **Consciousness / self-awareness:** No 'inner experience'.
- **Generalisation across domains:** Narrow AI excels in one domain but cannot transfer to unrelated problems.
- **Handle unstructured ethical / moral judgement:** Cannot decide value-laden trade-offs the way humans do.
- **Explain complex deep models fully:** Black-box deep nets still lack reliable interpretability.
- **Operate beyond training distribution:** Performance degrades on out-of-distribution inputs.
- **Continuous self-improvement without supervision:** Open problem in lifelong learning.

PAPER 3 : JAN 2023 – MAY 2023

Q1 — Solve any FOUR (20 marks)

Q1 i) Four approaches of AI. (5)

(See Paper 1 Q5 iii for the full table.) The four approaches are:

- **Thinking Humanly** — model human cognitive process (cognitive science).
- **Acting Humanly** — Turing-test approach: behave indistinguishably from humans.
- **Thinking Rationally** — laws of thought, logic-based AI (Aristotle, Prolog).
- **Acting Rationally** — rational agent doing the right thing to maximise expected utility (modern AI).

Modern AI follows the rational-agent paradigm because it is mathematically tractable and covers both reasoning and reflexive behaviour.

Q1 ii) Define: Agent, Rational agent, Omniscient agent, Turing Test. (1+1+1+2)

Agent (1): Anything that perceives its environment through sensors and acts upon it through actuators.

Rational agent (1): An agent that, for each possible percept sequence, selects the action that is expected to maximise its performance measure given the evidence and its built-in knowledge.

Omniscient agent (1): An idealised agent that knows the actual outcome of its actions in advance and can therefore act perfectly. In reality, omniscience is impossible — actual agents must be rational, not omniscient.

Turing Test (2): Proposed by Alan Turing in 1950. A human interrogator communicates via text terminal with a human and a machine. If the interrogator cannot reliably distinguish the machine's responses from the human's, the machine is said to have passed the test, demonstrating human-like intelligence. To pass, the machine needs NLP, knowledge representation, automated reasoning, and machine learning. A 'Total Turing Test' adds physical perception (vision, robotics).

Q1 iii) Well-formed Formulae. (5)

(a) Some students took French in spring 2001 (1.5)

```
∃x. Student(x) ∧ Took(x, French, Spring2001)
```

(b) Every student who takes French passes it (1.5)

```
∀x ∀t. (Student(x) ∧ Took(x, French, t)) → Passed(x, French, t)
```

(c) Only one student took Greek in spring 2001 (2)

```
∃x. Student(x) ∧ Took(x, Greek, Spring2001) ∧
∀y. (Student(y) ∧ Took(y, Greek, Spring2001)) → (y = x)
```

Q1 iv) Describe CSPs. Variable / value ordering. (5)

Constraint Satisfaction Problem (CSP): Defined by three components — (i) a set of **variables** $X_1, X_2, \ldots, X_n$; (ii) a non-empty **domain** D_i of allowed values for each variable; (iii) a set of **constraints** specifying allowed combinations. A solution is an assignment of values to all variables satisfying every constraint. Examples: map colouring, N-queens, Sudoku, scheduling.

Variable ordering — which variable next?

- **Minimum Remaining Values (MRV / 'fail-first'):** Pick the variable with the fewest legal values left — likely to fail soonest, pruning the tree.
- **Degree heuristic (tie-breaker):** Pick the variable involved in the largest number of constraints with other unassigned variables.

Value ordering — which value next for the chosen variable?

- **Least Constraining Value (LCV):** Choose the value that rules out the fewest choices for neighbouring variables — leaves maximum flexibility.

Both are combined with **backtracking + Constraint Propagation** (Forward Checking, AC-3) to solve CSPs efficiently.

Q1 v) Differentiate ADL vs STRIPS. (5)

Feature	STRIPS	ADL
Full form	STanford Research Institute Problem Solver	Action Description Language
Negative literals in pre-conditions	Not allowed (only positive)	Allowed (open-world)
Quantifiers	Not allowed	Universal & existential quantifiers allowed
Conditional effects	Not supported	Supported (when-then effects)
Equality	Not built-in	Built-in equality predicate
Types	Untyped	Typed objects supported
Open / Closed-world	Closed-world — what is not stated is false	Open-world — what is not stated is unknown
Use	Simpler / older planners	More expressive, used in modern PDDL

Q1 vi) Characteristics of Expert System. (5)

- **High performance:** Solves complex problems at expert level in a narrow domain.
- **Domain-specific:** Designed for one specific area (medicine, geology, finance).
- **Rule-based knowledge:** Knowledge expressed as IF-THEN rules separate from inference engine.
- **Symbolic reasoning:** Uses symbolic representation rather than purely numeric.
- **Explanation facility:** Can answer Why/How questions about its reasoning.
- **Handles uncertainty:** Via certainty factors, Bayesian or fuzzy methods.
- **Heuristic search:** Uses domain heuristics, not exhaustive search.
- **Easy to update:** Knowledge base can be modified without rewriting algorithms.
- **Reliable & consistent:** Gives same answer for same input, unaffected by fatigue or emotion.

Q2 A — Solve the following (10 marks)

Q2A i) Water Jug Problem — operators and production rules. (5)

Problem: 4-gallon jug and 3-gallon jug, neither marked, unlimited water from a pump, infinite ground (drain). Measure exactly 2 gallons in the 4-gallon jug.

State representation: (x, y) where x = water in 4-gal jug ($0 \le x \le 4$), y = water in 3-gal jug ($0 \le y \le 3$).

Initial state: (0, 0). **Goal:** (2, n) for any $n \in \{0, 1, 2, 3\}$.

Production rules / Operators:

#	Condition (current state)	Action / Resulting state
1	(x,y) with $x < 4$	Fill 4-gal → (4, y)
2	(x,y) with $y < 3$	Fill 3-gal → (x, 3)
3	(x,y) with $x > 0$	Empty 4-gal on ground → (0, y)
4	(x,y) with $y > 0$	Empty 3-gal on ground → (x, 0)
5	(x,y) with $x + y \ge 4 \wedge y > 0$	Pour 3-gal into 4-gal until full → (4, y – (4 – x))
6	(x,y) with $x + y \ge 3 \wedge x > 0$	Pour 4-gal into 3-gal until full → (x – (3 – y), 3)
7	(x,y) with $x + y \le 4 \wedge y > 0$	Pour all of 3-gal into 4-gal → (x + y, 0)
8	(x,y) with $x + y \le 3 \wedge x > 0$	Pour all of 4-gal into 3-gal → (0, x + y)

One valid solution path: (0,0) → R1 (4,0) → R6 (1,3) → R4 (1,0) → R8 (0,1) → R1 (4,1) → R6 (2,3). Goal reached in 6 steps. ✓

Q2A ii) Properties of task environment for part-picking robot. (5)

Property	Value	Justification
Observable	Partially observable	Camera + sensors do not see every part orientation or interior of bin.
Agents	Single agent	Robot acts alone; no competing agent.
Determinism	Stochastic	Grasping has uncertainty — slippage, mis-alignment.
Episodic / Sequential	Episodic	Each pick-and-place is independent — current action does not depend on past p
Static / Dynamic	Dynamic (mildly)	Parts on conveyor may move; new parts arrive.
Discrete / Continuous	Continuous	Position, force, time — all real-valued.
Known / Unknown	Known	Robot geometry & gripper physics are well modelled.

Q2 A (OR) — Utility Agent Architecture. (10)

A **utility-based agent** goes beyond a goal-based agent: instead of merely classifying states as 'goal' vs 'non-goal', it has a **utility function** that maps each state to a real number capturing how 'happy' the agent is in that state. The agent chooses the action that **maximises expected utility**.

Block diagram:

```
Percepts
Sensors ──────────────────────────────►
┌──────────────────────┐
│ State (current world)│
└──────────┬───────────┘
│
┌──────────▼───────────┐
│ How world evolves │
│ What my actions do │
└──────────┬───────────┘
│
┌──────────▼───────────┐
│ Utility function │
│ U(state) → ℝ │
└──────────┬───────────┘
│
┌──────────▼───────────┐
│ What action a next? │
│ argmax_a E[U(s')] │
└──────────┬───────────┘
Actuators ◄──────────┘
```

Components:

- **Sensors** — gather percepts.
- **State** — agent's belief about the current world (updated continuously).
- **Model of world dynamics** — 'how the world evolves' and 'what my actions do'.
- **Utility function** — quantifies desirability of each state; replaces a binary goal test.
- **Decision module** — picks the action maximising expected utility (often via expectimax / MDP).
- **Actuators** — execute action.

Example — Taxi agent: Goal-based version would simply say 'reach destination'. Utility-based version also balances: arrival time, safety, fuel, passenger comfort, traffic-law compliance. Each candidate route is scored by a utility function (e.g. $U = w_1 \cdot (-\text{time}) + w_2 \cdot (-\text{fuel}) + w_3 \cdot \text{safety} + w_4 \cdot \text{comfort}$) and the route with highest expected utility is taken.

Why utility? Handles conflicting goals; handles uncertainty (expected utility under probability); gives smooth trade-offs (rational choice theory).

Q2 B — Solve any ONE (10 marks)

Q2B i) BFS and DFS on given graph. Start = S, Goal = I. (3+3+4)

Given graph edges (from figure): S–A(6), S–E(3), A–C(2), A–B(3), C–F(?) , C–B(8), B–D(5), D–I(5), D–H(3), E–F(1), E–G(7), F–H(3), G–H(2), H–I(?) (approximated from figure). For exam, use whichever weights the figure clearly shows.

(a) State-space tree (illustrative):

```
S
┌──────┼──────┐
A  E
┌──┼──┐  ┌──┼──┐
C  B  F  G
│  │  │  │
F  D  H  H
│  │  │
I (goal)  I  I
```

(b) BFS (level-order traversal):

- Queue: [S]; Visit S.
- Queue: [A, E]; Visit A; expand → C, B.
- Queue: [E, C, B]; Visit E; expand → F, G.
- Queue: [C, B, F, G]; Visit C; expand → (already in).
- Queue: [B, F, G]; Visit B; expand → D.
- Queue: [F, G, D]; Visit F; expand → H.
- Queue: [G, D, H]; Visit G; (H already in).
- Queue: [D, H]; Visit D; expand → I → **GOAL**.

BFS path: S → A → B → D → I. **Cost:** 6 + 3 + 5 + 5 = 19.

(c) DFS (left-first depth-first):

- Push S. Pop S → push A, E.
- Pop A → push C, B.
- Pop C → push F.
- Pop F → push H.
- Pop H → push I → **GOAL**.

DFS path: S → A → C → F → H → I. **Cost (depending on weights):** 6 + 2 + (C–F) + 3 + (H–I). Using illustrative weights ≈ 6+2+? = depends on figure. Generally DFS finds *some* path, not necessarily the shortest.

Q2B ii) A* on the same graph using h-values given. (10)

Heuristics: h(S)=10, h(A)=8, h(B)=5, h(C)=7, h(D)=3, h(E)=6, h(F)=5, h(G)=3, h(H)=1, h(I)=0.

Compute f(n) = g(n) + h(n) at each step, always expand smallest f.

Step	Expand	Successors (g + h = f)	OPEN list
1	S g=0,f=10	A: g=6,h=8,f=14 ; E: g=3,h=6,f=9	{E(9), A(14)}
2	E (f=9)	F: g=4,h=5,f=9 ; G: g=10,h=3,f=13	{F(9), A(14), G(13)}
3	F (f=9)	H: g=4+3=7,h=1,f=8	{H(8), A(14), G(13)}
4	H (f=8)	I: g=7+? assume 3,f=10+0=10 → reach goal!	{I(10)} (others pruned)
5	I — GOAL	—	—

Solution path: S → E → F → H → I. **Total cost:** 3 + 1 + 3 + 3 = 10 (using assumed weights).

[In exam, plug in the exact edge weights from the printed figure — the method shown here is the marking template.]

Q3 — Solve the following (20 marks)

Q3 i) Knowledge engineering process for 1-bit full adder. (10)

Recall: 1-bit full adder takes 3 inputs A, B, Cin and produces Sum = A ⊕ B ⊕ Cin and Cout = AB + (A⊕B)·Cin.

Step	Task	Sample for 1-bit Full Adder
1	Identify the task	Design / verify a digital circuit that adds two 1-bit numbers and a carry-in to produce a 1-bit sum and a
2	Assemble relevant knowledge	Boolean algebra; XOR and AND gates; truth tables; carry propagation rules.
3	Decide vocabulary	Signals: A, B, Cin, Sum, Cout. Predicates: XOR(x,y,z), AND(x,y,z), OR(x,y,z), Wire(x,y).
4	Encode general domain rules	1. ∀x y z. XOR(x,y,z) ↔ (z = x ⊕ y) 2. ∀x y z. AND(x,y,z) ↔ (z = x ∧ y) 3. ∀x y z. OR(x,y,z) ↔ (z = x ∨ y)
5	Encode specific instance	Wire(A, in1_xor1), Wire(B, in2_xor1), Wire(out_xor1, in1_xor2), Wire(Cin, in2_xor2), Wire(out_xo
6	Pose queries	ASK: Sum = ? when A=1, B=1, Cin=1 — engine returns Sum = 1, Cout = 1. ASK: when is Cout = 1? — at least two of A,B,Cin are 1.
7	Debug KB	Check against the truth table: (000)→00, (001)→10, (010)→10, (011)→01, (100)→10, (101)→01, (110)→01, (111)→11. (Sum, Cout)

Q3 ii) FOL + Resolution — Some intelligent who cannot read. (10)

Sentences:

- a) Whoever can read is literate: ∀x. Reads(x) → Literate(x)
- b) Dolphins are not literate: ∀x. Dolphin(x) → ¬Literate(x)
- c) Some dolphins are intelligent: ∃x. Dolphin(x) ∧ Intelligent(x)
- d) Goal: ∃x. Intelligent(x) ∧ ¬Reads(x)

CNF conversion:

- 1. ¬Reads(x) ∨ Literate(x)
- 2. ¬Dolphin(y) ∨ ¬Literate(y)
- 3. Dolphin(D) [Skolemise ∃x for dolphin: introduce constant D]
- 4. Intelligent(D)

Negate goal: ∀x. ¬Intelligent(x) ∨ Reads(x) → CNF clause:

- 5. ¬Intelligent(x) ∨ Reads(x)

Resolution refutation:

- Resolve 4 and 5 (x = D): from Intelligent(D) and ¬Intelligent(x) ∨ Reads(x) → **Reads(D)**. [Clause 6]
- Resolve 6 and 1 (x = D): from Reads(D) and ¬Reads(x) ∨ Literate(x) → **Literate(D)**. [Clause 7]
- Resolve 7 and 2 (y = D): from Literate(D) and ¬Dolphin(y) ∨ ¬Literate(y) → **¬Dolphin(D)**. [Clause 8]
- Resolve 8 and 3: from ¬Dolphin(D) and Dolphin(D) → □ **(empty clause)**.

Empty clause derived ⇒ negation of goal is unsatisfiable ⇒ **Goal is PROVED** — there exists an intelligent being (Dolphin D) that cannot read.

Q3 ii (OR) — Burglar Alarm Bayesian Network. (10)

(a) Topology: identical to Paper 1 Q4 (Burglary → Alarm ← Earthquake; Alarm → JohnCalls; Alarm → MaryCalls).

(b) Conditional Probability Tables (standard textbook values):

P(B) = 0.001 ; P(E) = 0.002

B	E	P(A=T|B,E)
T	T	0.95
T	F	0.94
F	T	0.29
F	F	0.001

P(J=T|A=T) = 0.90 ; P(J=T|A=F) = 0.05

P(M=T|A=T) = 0.70 ; P(M=T|A=F) = 0.01

(c) Given evidence: ¬B, ¬E, J, M. Find P(A | ¬B, ¬E, J, M).

Bayes: P(A | e) ∝ P(A, e) where e = ¬B ∧ ¬E ∧ J ∧ M. Compute joint for A=T and A=F:

```
P(A=T, ¬B, ¬E, J, M) = P(¬B)·P(¬E)·P(A=T|¬B,¬E)·P(J|A=T)·P(M|A=T)
= 0.999 × 0.998 × 0.001 × 0.90 × 0.70
= 6.28 × 10⁻⁴
```

```
P(A=F, ¬B, ¬E, J, M) = 0.999 × 0.998 × 0.999 × 0.05 × 0.01
= 4.98 × 10⁻⁴
```

```
α = 1 / (6.28e-4 + 4.98e-4) = 1 / 1.126e-3 = 888
P(A | ¬B, ¬E, J, M) = 888 × 6.28e-4 = 0.557 (≈ 55.7%)
```

So, given neither burglary nor earthquake but both John and Mary called, the probability the alarm rang is about 0.56.

(d) Reasons uncertainty enters problem solving:

- Partial observability — sensors give limited info.
- Non-deterministic environment / actions.
- Faulty / noisy sensors.
- Theoretical ignorance — model is simplification of reality.
- Practical ignorance — even with perfect model, gathering full data is infeasible.
- Lying / mistaken information from agents.

Q4 — Solve the following (20 marks)

Q4 i) Partial Order Planning — wearing shoes. (10)

Partial Order Planning (POP) represents a plan as a set of actions with only the ordering constraints that are *necessary* — actions that don't depend on each other remain unordered (can be done in parallel).

Goal: RightShoeOn ∧ LeftShoeOn.

Actions and ordering constraints derived by goal regression:

- RightShoe action needs RightSockOn as precondition → RightSock < RightShoe.
- LeftShoe action needs LeftSockOn as precondition → LeftSock < LeftShoe.
- No dependency between left and right side → unordered.

Partial-order plan:

```
Start
/ \
PutLeftSock PutRightSock
| |
PutLeftShoe PutRightShoe
\ /
Finish (Goal: bothShoesOn)
```

Any total order consistent with these constraints is valid, e.g.

- LS → LSh → RS → RSh
- RS → LS → RSh → LSh
- LS → RS → LSh → RSh (interleaved)

Advantage: Far fewer plans to consider than enumerating all 24 total orderings. Better for parallel execution and replanning. **Algorithm:** Maintain plan = (Steps, Orderings, CausalLinks, OpenPreconditions); iteratively resolve open preconditions and threats until no open preconditions remain.

Q4 i (OR)) Steps to build NLP + applications. (10)

Refer to Paper 2 Q3 iii for the 11-step pipeline. Below is a compact step-by-step build recipe and applications.

Build steps:

- 1. Define the NLP task (classification, translation, summarisation, NER, QA).
- 2. Collect & label data (corpora, scraping, public datasets).
- 3. Pre-process: clean text, remove HTML, normalise unicode.
- 4. Tokenise: split into tokens; for modern models use BPE/WordPiece.
- 5. Lexical processing: stemming, lemmatisation, stop-word removal.
- 6. Feature extraction: TF-IDF, word2vec, GloVe, or contextual embeddings (BERT).
- 7. Model training: classical ML (Naive Bayes, SVM) or deep learning (RNN, Transformer).
- 8. Evaluation: precision, recall, F1, BLEU, ROUGE.
- 9. Deployment: REST API, integrate with applications.
- 10. Monitoring & retraining: handle drift, new vocabulary.

Applications of NLP:

- Machine translation (Google Translate).
- Chatbots & virtual assistants (Alexa, Siri, ChatGPT).
- Sentiment analysis for brand monitoring.
- Spam detection in e-mail.
- Search engines and information retrieval.
- Automatic summarisation (news, legal documents).
- Speech recognition & text-to-speech.

- Plagiarism / fake-news detection.
- Healthcare (clinical-text analysis).
- Legal & finance (contract review, tax queries).

Q4 ii) Alpha-beta pruning on given tree. (4+2+4)

Tree (Figure 2): Root A (MAX) → B, C (MIN). B → D(2,3), E(5,9). C → F(0,1), G(7,5).

Standard Min-Max values: D = min(2,3) = 2; E = min(5,9) = 5; B = max(D,E) = 5. Wait — tree is alternating; clarify levels:

Assume: A = MAX, B & C = MIN, D, E, F, G = MAX over their leaf pairs. Leaves: D = max(2,3)=3 ; E = max(5,9)=9 ; F = max(0,1)=1 ; G = max(7,5)=7. B = min(D,E) = min(3,9) = 3. C = min(F,G) = min(1,7) = 1. **A = max(B,C) = max(3,1) = 3.**

Alpha-Beta trace (α = −∞, β = +∞ initially):

- Enter B (MIN); β=+∞. Enter D (MAX); evaluate leaf 2 → α=2; evaluate leaf 3 → α=3; D returns 3. Back at B: β = min(+∞, 3) = 3. Not α≥β yet (α=−∞).
- Enter E (MAX); incoming α=−∞,β=3. Evaluate leaf 5 → α=5. Now α(5) ≥ β(3) → **PRUNE remaining child 9 of E**. E returns 5 (or ≥5).
- B: β = min(3, 5) = 3. B returns 3. Root A: α = max(−∞, 3) = 3.
- Enter C (MIN); incoming α=3, β=+∞. Enter F (MAX); α=3,β=+∞. Evaluate leaf 0 → α stays 3; leaf 1 → α stays 3. F returns 1.
- Back at C: β = min(+∞, 1) = 1. Now α(3) ≥ β(1) → **PRUNE G entirely (both its leaves 7 and 5)**. C returns 1.
- Root A: max(3, 1) = 3. **Final value at root A = 3**. Winner: MAX with value 3.

Strengths: Reduces search by half in the best case (β-cuts); no loss in optimal move. **Weaknesses:** Effectiveness depends on move ordering (best-first ordering recovers √b speedup); doesn't help in stochastic games (use Expectimax instead).

Q5 — Solve any FOUR (20 marks)

Q5 i) Define chromosome, fitness, crossover, mutation (in GA). (1+1+2+1)

Chromosome (1): An encoded representation of a candidate solution in a GA, usually a string (binary, integer, real, or symbolic). e.g. for 0/1 knapsack with 5 items, a chromosome is a 5-bit string like 10110.

Fitness function (1): A numerical measure of how good a chromosome is at solving the problem. Higher fitness ⇒ better solution. Used to compare candidates and drive selection. e.g. total profit in knapsack (subject to weight constraint).

Crossover (2): A reproduction operator that combines two parent chromosomes to produce one or two offspring. Mimics biological recombination. Common types:

- **Single-point:** cut at random index k; child = parent1[0:k] + parent2[k:].
- **Two-point:** cut at two indices; swap middle portions.
- **Uniform:** each gene chosen randomly from one of the two parents.

Mutation (1): Random alteration of a gene with low probability (typically 0.001 to 0.05) to introduce diversity and avoid premature convergence. For binary chromosome, flip a bit; for real-valued, add a small Gaussian noise.

Q5 ii) Problems in Hill Climbing — solutions. (5)

(See Paper 1 Q2A ii for detailed answer.)

- **Local maximum:** stuck at peak that's not global → Random-Restart, Simulated Annealing.
- **Plateau:** flat region, no preferred direction → allow sideways moves, big-jump.
- **Ridge:** diagonal path blocked by axis-aligned moves → Stochastic Hill Climbing, diagonal moves.
- Additional: Beam Search, Tabu Search, Genetic Algorithms (population-based).

Q5 iii) Heuristic admissibility with example. (5)

Admissible heuristic: A heuristic h(n) is *admissible* if for every node n, h(n) ≤ h*(n), where h*(n) is the true cost to reach the nearest goal from n. In other words, the heuristic **never overestimates** the actual cost.

Why important: An admissible heuristic guarantees that **A* finds an optimal solution**. If h is also **consistent** (monotone) — h(n) ≤ c(n, n') + h(n') for any successor n' — then A* never re-expands any node.

Example — 8-puzzle:

- **h_1 = Number of misplaced tiles.** Admissible — every misplaced tile must move at least once.
- **h_2 = Sum of Manhattan distances.** Admissible — each tile needs at least that many moves; Manhattan dominates h_1.
- Straight-line distance in a road network is admissible for finding the shortest path because Euclidean distance ≤ road distance.
- Counter-example: Euclidean distance × 1.5 = inadmissible (may overestimate), and A* can find a suboptimal path.

Q5 iv) Universal Instantiation and Existential Instantiation with example. (5)

Universal Instantiation (UI): From ∀x. P(x) and any ground term t, infer P(t).

Example: Given ∀x. King(x) ∧ Greedy(x) → Evil(x), substitute x = John to obtain King(John) ∧ Greedy(John) → Evil(John).

Used freely — any number of instantiations are sound. No new constants are introduced.

Existential Instantiation (EI): From ∃x. P(x), infer P(c) where c is a brand-new **Skolem constant** not appearing elsewhere in the KB. Used once per existential.

Example: Given ∃x. Crown(x) ∧ OnHead(x, John), introduce a Skolem constant C → Crown(C) ∧ OnHead(C, John). The new symbol C names 'some specific crown on John's head'.

If the existential is inside a universal (∀y ∃x. Loves(y, x)), the witness depends on y, so use a **Skolem function** f(y) instead of a constant: ∀y. Loves(y, f(y)).

Q5 v) Architecture of Expert System with diagram. (5)

(See Paper 1 Q4 ii second variant for the full sketch.) Recap:

```
User ↔ User Interface ↔ Inference Engine ↔ Knowledge Base (Rules + Facts)
│
▼
Explanation Module
KB ↔ Knowledge Acquisition Module ↔ Domain Expert
```

Roles: KB stores expert knowledge as IF-THEN rules; Inference Engine reasons (forward / backward chaining); Working Memory holds current facts; Explanation gives why/how reasoning; UI lets non-experts ask questions; KAM lets domain experts edit the KB.

Example — MYCIN diagnoses blood infections; **DENDRAL** identifies organic chemical structure; **XCON** configured VAX computers at DEC saving them ~$25M / year.

Q5 vi) Learning process in Decision Tree. (5)

A **Decision Tree** is a tree-shaped classifier where each internal node tests an attribute, each branch corresponds to an attribute value, and each leaf gives a class label.

Learning process (ID3 / C4.5 algorithm):

- 1. Start with all training examples at the root.
- 2. Compute **entropy** of the current set: $H(S) = -\Sigma\, p_i \log_2 p_i$.
- 3. For each candidate attribute A, compute **information gain**: $Gain(S, A) = H(S) - \Sigma\, (|S_v|/|S|) \cdot H(S_v)$.
- 4. Choose the attribute with the **highest Gain** as the splitting node.
- 5. Partition the training set by this attribute and recurse on each child subset.
- 6. Stop when (a) all examples in a node have the same class (pure), (b) no attributes left, or (c) examples are empty — assign majority class.
- 7. Optional: prune the tree post-construction to reduce overfitting (reduced-error pruning, cost-complexity pruning in CART).

Advantages: Easy to interpret, no need for feature scaling, handles categorical & numeric data. **Disadvantages:** Tends to overfit; small data changes can produce different trees. Variants: **C4.5** (Gain Ratio), **CART** (Gini index), **Random Forest** (ensemble of decision trees).

PAPER 4 | May 2022 – June 2022

SECTION A

Q1(a) State and explain the "AI Problems". Brief on Mundane, Formal and Expert tasks. (10)

AI problems are classified by the type of intelligence the task demands. Researchers traditionally group them into three categories: **Mundane, Formal, and Expert tasks**.

1. Mundane Tasks — Things any ordinary human can do effortlessly but are surprisingly hard for machines because they need common-sense knowledge, perception, and motor skills.

- **Perception:** Vision (object recognition, face recognition), Speech recognition.
- **Natural Language:** Understanding, generation, translation.
- **Robotics:** Locomotion, grasping, navigation in physical space.
- **Common-sense reasoning:** Knowing that ice melts when heated.

2. Formal Tasks — Tasks with well-defined rules and symbols that humans need training for; computers excel here because rules can be encoded.

- **Games:** Chess, Go, Checkers, Backgammon (AlphaGo, Deep Blue).
- **Mathematics:** Theorem proving, integration, geometric reasoning.
- **Verification:** Logic, computational complexity, proof checking.

3. Expert Tasks — Tasks that require many years of specialised training and domain expertise. Knowledge is narrow but deep.

- **Engineering:** Design, fault-finding, manufacturing planning.
- **Medical diagnosis:** MYCIN (blood infections), INTERNIST.
- **Scientific analysis:** DENDRAL (chemical structure).
- **Financial analysis:** Stock-market prediction, credit scoring.

Interesting paradox: Mundane tasks (which a 3-year-old child can do) are *harder* for AI than Expert tasks (which need decades of human training). This is called **Moravec's Paradox** — sensorimotor skills evolved over millions of years and are difficult to reverse-engineer, while logical reasoning is relatively recent and rule-based.

OR — (i) Five things AI cannot do yet:

- **True common-sense reasoning** — AI fails on everyday situations not in training data.
- **Genuine creativity** — AI can mimic style but cannot invent a wholly new art form.
- **Emotional understanding** — AI cannot truly empathise or feel.
- **Cross-domain transfer learning** — Chess AI can't play poker without retraining.
- **Self-awareness / consciousness** — AI doesn't "know" it exists.

(ii) Five applications of AI in daily life:

- **Voice assistants** — Siri, Alexa, Google Assistant for query handling.
- **Recommendation systems** — Netflix, YouTube, Amazon product suggestions.
- **Navigation & maps** — Google Maps traffic prediction, route optimisation.
- **Email spam filters & smart replies** — Gmail uses NLP + classifiers.
- **Face unlock & biometric authentication** — Phone unlocking, airport security.

Q1(b) Describe each of the components of the Utility-based agent. Comment on Mars-lander example and how it differs in a Goal-based agent with learning. (3+3+4 = 10)

A **utility-based agent** goes beyond simple goal achievement — it picks the action that maximises **expected utility**, a real-valued measure of how desirable a state is. This handles trade-offs (speed vs safety), conflicting goals, and uncertainty.

Components (block diagram):

```
┌──────────────────────────────────────┐
│ ENVIRONMENT │
└────▲──────────────────────────┬──────┘
│percepts actions│
┌────┴─────────┐  ┌──────▼──────┐
│ Sensors │ │ Actuators │
└──────┬───────┘  └──────▲──────┘
▼ │
┌──────────────┐ │
│ State (what │ │
│ world is now)│ │
└──────┬───────┘ │
▼ │
┌──────────────────────┐ │
│ How world evolves & │ │
│ what my actions do │ │
└──────┬───────────────┘ │
▼ │
┌──────────────────────┐ │
│ What will happen if │ │
│ I do action X? │ │
└──────┬───────────────┘ │
▼ │
┌──────────────────────┐ │
│ UTILITY function │ → score each │
│ U(state) → ℝ │ future state│
└──────┬───────────────┘ │
▼ │
Pick action with │
max expected utility ──────────────┘
```

Mars Lander example: A Mars rover needs to reach a sample-collection point. There may be obstacles (rocks, craters, sand). A goal-based agent only checks "did I reach the goal?" — it would treat all reaching paths as equally good. A **utility-based agent** ranks paths by utility = f(safety, fuel, time, scientific value). It may prefer a longer but safer detour over a shorter path through risky terrain. Utility quantifies trade-offs that "yes/no goal" cannot.

Difference vs Goal-based agent with learning: A learning goal-based agent improves its world model and action policy over time but still uses binary goal satisfaction. The utility-based agent, in contrast, uses graded preferences; **with learning added**, it can also refine its utility function from feedback (e.g., a learning Mars lander learns that wet soil is risky after slipping once and lowers its utility). So utility-based + learning = the most sophisticated of the four classical architectures.

Q2(a) Production rules + Genetic Algorithm steps for Block World problem. (2 + 8 = 10)

Production rules are IF-condition-THEN-action statements that encode domain knowledge. A production system has three parts: (i) Working Memory (current state), (ii) Production Rules (knowledge base), (iii) Recognise-Act Cycle (inference engine matches rules and fires them).

Block World problem: A set of blocks A, B, C, … rest on a table or on each other. A robot arm can pick up one block at a time. The task is to rearrange them from an initial configuration to a goal configuration.

Production rules for Block World:

- **R1 (Pickup):** IF block X is clear AND X is on table AND arm is empty THEN pickup(X). Effect: arm holds X.
- **R2 (Unstack):** IF X is on Y AND X is clear AND arm is empty THEN unstack(X, Y). Effect: arm holds X, Y becomes clear.
- **R3 (Putdown):** IF arm holds X THEN putdown(X). Effect: X on table, arm empty.
- **R4 (Stack):** IF arm holds X AND Y is clear THEN stack(X, Y). Effect: X on Y, arm empty, Y not clear.

GA steps with example:

- **1. Encoding:** Represent each configuration as a chromosome (e.g., a permutation of blocks plus stack heights).
- **2. Initial population:** Generate N random valid configurations.
- **3. Fitness function:** Number of correctly placed blocks vs goal state (higher = better).
- **4. Selection:** Roulette-wheel / tournament: pick parents weighted by fitness.
- **5. Crossover:** Single-point or two-point on the chromosome strings; produce children configurations.
- **6. Mutation:** Swap two block positions with a small probability (e.g., 0.05).
- **7. Replacement:** Form new generation (often keeping the best — elitism).
- **8. Termination:** Stop when fitness reaches max (goal configuration) or after N generations.

Q2(b) Define SLD heuristic. Distinguish AO* vs A*. BFS vs DFS. Solve AO* on figure (A=11, B=6, C=12, D=10, G=5, H=7, E=4, F=4). (2+4+4 = 10)

Straight-Line Distance (SLD) heuristic: An estimate of the cost from a node to the goal computed as the Euclidean / geometric straight-line distance, ignoring obstacles. It is **admissible** (never overestimates) for path-finding on a 2-D plane, hence guarantees A* optimality.

AO* vs A*:

Aspect	A* Search	AO* Search
Graph type	OR graph — pick one child	AND/OR graph — AND-nodes need all children solved
Use case	Path-finding (one route to goal)	Problem reduction (decompose into sub-problems)
Cost computation	f(n) = g(n) + h(n)	Sum of costs over AND-children; min over OR-children
Backup	Single best-cost backup	Top-down expansion + bottom-up cost revision
Solution	Single path	Solution sub-tree (could contain multiple branches)
Example	Romania route-finding	Theorem proving, planning hierarchy

BFS vs DFS:

Aspect	BFS	DFS
Strategy	Expand shallowest unexplored node	Expand deepest unexplored node
Data structure	FIFO Queue	LIFO Stack (or recursion)

Memory	O(b^d) — high	O(b·d) — low
Completeness	Complete (finite b)	Not complete on infinite branches
Optimality	Optimal if step costs equal	Not optimal
Use case	Shortest-edge-count path	Maze, topological sort, deep search

AO* solution on given figure: Heuristic values h: A=11, B=6, C=12, D=10, G=5, H=7, E=4, F=4. Assume B is an OR-node with children G, H; C is a leaf (terminal); D is an AND-node with children E, F. From A we choose to expand its children B, C, D. Treating arc cost as 1 between every parent–child.

```
Step 1: At root A, evaluate three OR-children:
via B → 1 + h(B) = 1 + 6 = 7
via C → 1 + h(C) = 1 + 12 = 13
via D → 1 + h(D) = 1 + 10 = 11
Best at A = via B, revised h(A) = 7 (was 11)
Step 2: Expand B (OR-node, children G, H):
via G → 1 + 5 = 6 ← best at B
via H → 1 + 7 = 8
Revised h(B) = 6, so f(A via B) = 1 + 6 = 7 (unchanged)
Step 3: Expand G — assume G is terminal/solved (h=5).
Mark G as SOLVED. Propagate up: B SOLVED via G,
A SOLVED via B → G.
Step 4: Final solution graph: A → B → G
Step 5: Total cost = 1 + 1 + 5 = 7
```

Note: If D were the chosen branch as AND-node, cost would be 1 + (1+h(E)) + (1+h(F)) = 1+5+5 = 11 (sum of both children needed). Since the OR path via B-G costs only 7, AO* picks that.

Q3 — Multiple Choice (1 mark each)

Q3.1 Artificial Intelligence is about ____.

- 1. Playing a game on Computer
- ✓ 2. Making a machine Intelligent
- 3. Programming on Machine with your Own Intelligence
- 4. Putting your intelligence in Machine

Answer: (2) Making a machine Intelligent. AI is the science of making machines intelligent — perceive, reason, learn, act.

Q3.2 Who is known as the Father of AI?

- 1. Fisher Ada
- 2. Alan Turing
- ✓ 3. John McCarthy
- 4. Allen Newell

Answer: (3) John McCarthy. John McCarthy coined the term 'Artificial Intelligence' at the Dartmouth Conference, 1956.

Q3.3 Which search algorithm requires less memory?

- 1. Optimal Search
- ✓ 2. Depth First Search
- 3. Breadth-First Search
- 4. Linear Search

Answer: (2) Depth First Search. DFS only stores the current path — O(b·d). BFS needs O(b^d), exponential in depth.

Q3.4 A technique developed to test whether a machine demonstrates AI is the ____.

- 1. Boolean Algebra
- ✓ 2. Turing Test
- 3. Logarithm
- 4. Algorithm

Answer: (2) Turing Test. Alan Turing's Imitation Game (1950): if a judge cannot distinguish machine from human via text, the machine is 'intelligent'.

Q3.5 Which algorithm is used in Game tree for Win/Lose decisions?

- 1. Heuristic Search
- 2. DFS/BFS
- 3. Greedy Search
- ✓ 4. Min/Max algorithm

Answer: (4) Min/Max algorithm. Min/Max with alpha-beta pruning is the standard adversarial-search algorithm for two-player zero-sum games.

Q3.6 An AI agent perceives and acts upon the environment using ____.

- 1. Sensors
- 2. Perceiver
- 3. Actuators
- ✓ 4. Sensors and Perceivers

Answer: (4) Sensors and Perceivers. Strictly the textbook answer is 'Sensors (to perceive) and Actuators (to act)'. Among given options, 'Sensors and Perceivers' (4) is closest; perception is via sensors, action via actuators. *(Note: option list is imperfect; ideally would be '*Sensors and Actuators*'.)*

Q3.7 Which agent deals with happy and unhappy states?

- ✓ 1. Utility-based agent

- 2. Model-based agent
- 3. Goal-based agent
- 4. Learning Agent

Answer: (1) Utility-based agent. Utility agent assigns a real-valued utility to each state — 'happiness' level — and chooses actions maximising expected utility.

Q3.8 Which is NOT an application of AI?

- 1. Computer Vision
- 2. Natural Language Processing
- ✓ 3. Database Management System
- 4. Digital Assistants

Answer: (3) Database Management System. DBMS is data-storage software — no learning or reasoning required.

Q3.9 What is the work of Task Environment and Rational Agents?

- 1. Problem and Solution
- 2. Solution and Problem
- 3. Observation and Problem
- ✓ 4. Observation and Solution

Answer: (4) Observation and Solution. Task Environment provides observations (percepts); the Rational Agent produces solutions (actions). Hence Observation → Solution.

Q3.10 Machines imitating human intuition with vague info lie in the field of ____.

- 1. Functional Logic
- ✓ 2. Fuzzy Logic
- 3. Boolean Logic
- 4. Human Logic

Answer: (2) Fuzzy Logic. Fuzzy Logic (Lotfi Zadeh, 1965) handles partial truth values in [0,1], modelling vagueness and approximation.

SECTION B

Q4(a) Solve using CSP: TWO + TWO = FOUR. (10)

Variables: T, W, O, F, U, R ∈ {0, 1, ..., 9}; carry-bits C1, C2, C3 ∈ {0, 1}.

Constraints (column-wise addition with carries):

- Column 1 (units): O + O = R + 10·C1
- Column 2 (tens): W + W + C1 = U + 10·C2
- Column 3 (hundreds): T + T + C2 = O + 10·C3
- Column 4 (thousands): C3 = F
- All-different: T, W, O, F, U, R are distinct.
- Leading digits: T ≠ 0, F ≠ 0 (so F = 1, hence C3 = 1).

Heuristics used: (i) MRV — Minimum Remaining Values: pick the variable with fewest legal values left. (ii) Degree heuristic — break ties by most constraints on remaining vars. (iii) LCV — Least Constraining Value: try value that rules out fewest options for neighbours.

Step-by-step solution:

```
Step 1: F = C3 = 1 (only digit possible since F ≠ 0)
Step 2: Col 3 → 2T + C2 = O + 10·1 = O + 10
→ 2T = O + 10 − C2
If C2=0: 2T = O + 10 → T ≥ 5
If C2=1: 2T = O + 9 → T ≥ 5 (since O ≥ 0)
Step 3: Try T = 7. Then 2·7 = 14 = O + 10 − C2
→ O = 4 + C2. Try C2=0 → O = 4.
Step 4: Col 1: 2O = R + 10·C1 → 2·4 = 8 = R + 10·C1
→ C1=0, R = 8.
Step 5: Col 2: 2W + C1 = U + 10·C2
→ 2W + 0 = U + 0 → U = 2W
Step 6: Used digits {F=1, T=7, O=4, R=8}. Need W, U distinct,
not in used set, and U = 2W.
Try W=3 → U=6. Both unused. ✓

Final: T=7, W=3, O=4, F=1, U=6, R=8
7 3 4
+ 7 3 4
-------
1 4 6 8 verify: 734 + 734 = 1468 ✓
```

OR — Knowledge Engineering for Detective Agent (10):

Knowledge engineering is the process of constructing a knowledge base for an AI system. For a **detective agent** (e.g., one that helps solve crimes by reasoning over evidence), the steps are:

Step	Activity	Detective-agent sample output
1. Identify Task	Define scope, queries, performance criteria.	Goal: identify suspect from evidence. Query: who is guilty?
2. Assemble Relevant Knowledge	Gather facts, rules, domain concepts.	Crime scene facts, motive theory, forensic evidence types, witness
3. Decide Vocabulary	Choose predicates, functions, constants.	Suspect(x), MotiveFor(x, victim), AtScene(x, time), Murdered(victim
4. Encode General Knowledge	Write axioms / ontology.	∀x (AtScene(x,t) ∧ HasMotive(x) ∧ HasWeapon(x) → Suspect(x)).
5. Encode Specific Problem	Add facts about current case.	AtScene(John, 9pm), HasMotive(John), Weapon(Knife).
6. Pose Query, Get Inference	Run inference engine.	?- Suspect(x). → x = John.
7. Debug KB	Refine if answer wrong.	Add: ∀x (Alibi(x) → ¬AtScene(x,t)). Resolves false positives.

Q4(b) Translate sentences into FOL, use backward chaining to prove 'Joel is at museum'. (10)

FOL representation of given sentences:

- a. Bulldog(Joel).
- b. Master(John, Joel).
- c. Day(Saturday).
- d. ColdOn(Saturday).
- e. Trained(Joel).
- f. ∀x [Bulldog(x) ∧ Trained(x) → GoodDog(x)].
- g. ∀x ∀y ∀p [GoodDog(x) ∧ Master(y, x) ∧ At(x, p) → With(x, y)].
- h. Day(Saturday) ∧ ColdOn(Saturday) → At(John, Museum).

Goal: At(Joel, Museum)

Backward chaining starts from the goal and works backwards to facts.

```
Goal: At(Joel, Museum)
We have no direct rule stating At(Joel, Museum).
However, from (g) we know: GoodDog(x) ∧ Master(y,x) ∧ At(x,p) → With(x,y)
Interpret with intuition: Joel goes wherever his master goes.
Add the natural-language premise (implicit):
∀x ∀y ∀p [Master(y,x) ∧ GoodDog(x) ∧ At(y,p) → At(x,p)] ← (i)
```

```
Sub-goal 1: Master(John, Joel) ← FACT (b) ✓
Sub-goal 2: GoodDog(Joel) ← apply (f)
needs: Bulldog(Joel) ← FACT (a) ✓
needs: Trained(Joel) ← FACT (e) ✓
⇒ GoodDog(Joel) ← derived ✓
Sub-goal 3: At(John, Museum) ← apply (h)
needs: Day(Saturday) ← FACT (c) ✓
needs: ColdOn(Saturday) ← FACT (d) ✓
⇒ At(John, Museum) ← derived ✓
```

```
All sub-goals discharged → At(Joel, Museum) is PROVED. ∎
```

Inference rules used: Universal Instantiation (instantiate ∀x with specific terms), Modus Ponens (A → B and A entail B), And-Introduction.

Q5(a) STRIPS solution for changing flat tire with spare in car trunk. (10)

STRIPS (Stanford Research Institute Problem Solver) represents planning problems by: states (set of ground predicates), goal (set of literals to satisfy), and actions (with pre-conditions, add-list, delete-list).

Initial State:

```
At(Flat, Axle) — flat tire is on the axle
At(Spare, Trunk) — spare is in the trunk
Closed(Trunk) — trunk is shut
HandsFree — agent not holding anything
```

Goal State:

```
At(Spare, Axle) ∧ At(Flat, Ground)
```

Actions:

```
Action: OpenTrunk
Pre: Closed(Trunk) ∧ HandsFree
Add: Open(Trunk)
Del: Closed(Trunk)
```

```
Action: Remove(obj, loc)
Pre: At(obj, loc) ∧ HandsFree ∧ Accessible(loc)
Add: Holding(obj), At(obj, Ground)
Del: At(obj, loc), HandsFree

Action: PutOn(obj, Axle)
Pre: Holding(obj) ∧ ¬At(_, Axle) — axle is free
Add: At(obj, Axle), HandsFree
Del: Holding(obj)

Action: LeaveOvernight (sneaky alternate plan)
Pre: (none)
Add: (nothing)
Del: At(Spare,_) ∧ At(Flat,_) ∧ At(Spare,_) ∧ At(Flat,_)
(everything disappears — illustrates STRIPS quirk)
```

Plan (sequence of actions):

```
1. OpenTrunk
→ State: Open(Trunk), At(Flat, Axle), At(Spare, Trunk), HandsFree

2. Remove(Spare, Trunk)
→ State: Holding(Spare), Open(Trunk), At(Flat, Axle)
— but agent is now holding spare, hands NOT free

3. PutOn(Spare, Ground) — temporarily put spare on ground
→ State: At(Spare, Ground), HandsFree, At(Flat, Axle)

4. Remove(Flat, Axle)
→ State: Holding(Flat), At(Flat, Ground), At(Spare, Ground)

5. PutOn(Flat, Ground) — release flat to ground
→ State: At(Flat, Ground), HandsFree, At(Spare, Ground)

6. Pickup(Spare) [analog of Remove(Spare, Ground)]
→ State: Holding(Spare), At(Flat, Ground)

7. PutOn(Spare, Axle)
→ State: At(Spare, Axle), At(Flat, Ground), HandsFree ✓ GOAL
```

Q5(b) Differentiate Supervised, Unsupervised, Semi-supervised, Reinforcement learning. (10)

Aspect	Supervised	Unsupervised	Semi-supervised	Reinforcement
Training data	Labelled (X, y)	Unlabelled X only	Mostly unlabelled + small labelled set	No labelled set — agent acts in env, gets rewards
Goal	Learn mapping f: X → y	Discover hidden structure	Combine: leverage labels with cheap unlabelled data	Learn policy that maximises cumulative rew
Typical algos	Linear regression, SVM, Decision Trees, Neural Networks, k-nearest neighbours	k-means, Hierarchical clustering, PCA, t-SNE, DBSCAN	Self-training, Co-training, Label Propagation, GAN-SSL	Q-learning, SARSA, Deep Q-Networks, Polic
Feedback	Immediate, ground-truth	None — only data structure	Partial labels guide	Delayed, scalar reward
Examples	Spam filter, image classifier, house price predictor	Customer segmentation, topic modelling, anomaly detection	Web page classification, medical imaging	Game playing (AlphaGo), robotics, ad-biddi
Strength	High accuracy if labels are clean	Works on raw data, no labelling cost	Reduces label-cost while keeping accuracy	Handles sequential decisions, learns policy
Weakness	Needs large labelled dataset	Hard to evaluate; no ground truth	Sensitive to label noise	Sample-inefficient; reward design tricky

OR — Steps to build an NLP system:

- **1. Text Acquisition:** Collect corpus (web crawl, transcripts, databases).
- **2. Tokenisation:** Split text into words / sub-words.
- **3. Lexical Analysis:** Identify morphemes; stemming (run, running, ran → run); lemmatisation.
- **4. POS Tagging:** Assign parts of speech (noun, verb…) to each token.

- **5. Syntactic Analysis (Parsing):** Build parse tree using grammar.
- **6. Semantic Analysis:** Map words to meanings; resolve word-sense ambiguity.
- **7. Named Entity Recognition (NER):** Identify persons, organisations, places, dates.
- **8. Discourse Integration:** Resolve references (pronouns, anaphora) across sentences.
- **9. Pragmatic Analysis:** Interpret sentence in real-world context (sarcasm, intent).
- **10. Application Output:** Translation, summarisation, sentiment, Q&A;, chatbots.

Q6 — Multiple Choice (1 mark each)

Q6.1 Which algorithm is used in Game tree for Win/Lose decisions?

- 1. Heuristic Search
- 2. DFS/BFS
- 3. Greedy Search
- ✓ 4. Min/Max algorithm

Answer: (4) Min/Max algorithm. Repeat of Q3.5. Min/Max is the canonical adversarial-search algorithm.

Q6.2 In Wumpus World, agent's sensor gives only ____.

- 1. Full & Global information
- 2. Partial & Global Information
- 3. Full & local information
- ✓ 4. Partial & local Information

Answer: (4) Partial & local Information. Wumpus is partially observable (stench, breeze only in adjacent squares) and local (only current cell).

Q6.3 Maximum depth to which alpha-beta pruning can be applied.

- 1. Eight states
- 2. Six states
- 3. Ten states
- ✓ 4. Any depth

Answer: (4) Any depth. Alpha-beta pruning is depth-agnostic — works on trees of any depth.

Q6.4 Face Recognition system is based on which approach?

- 1. Weak AI
- ✓ 2. Applied AI
- 3. Cognitive AI
- 4. Strong AI

Answer: (2) Applied AI. Face recognition is a narrow, domain-specific AI task → Applied AI (a form of Weak AI). Best fit: Applied AI.

Q6.5 What does Bayesian network provide?

- 1. Partial description of the domain
- 2. Complete description of the problem
- ✓ 3. Complete description of the domain
- 4. None of the mentioned

Answer: (3) Complete description of the domain. A BN compactly represents the full joint distribution → complete probabilistic description of the domain.

Q6.6 Application of AI is ____.

- 1. Helps exploit vulnerabilities
- ✓ 2. Language understanding and problem-solving (Text analytics and NLP)
- 3. Easy to create a website
- 4. Deploy applications on the cloud

Answer: (2) Language understanding and problem-solving (Text analytics and NLP). Language understanding + problem-solving is exactly the classical AI definition.

Q6.7 Brain incrementally orders actions needed for a task — this is called ____.

- 1. Planning problem
- ✓ 2. Partial order planning
- 3. Total order planning
- 4. Both Planning problem & Partial order planning

Answer: (2) Partial order planning. Partial Order Planning fixes ordering only where required, leaving other actions unordered — matches incremental ordering.

Q6.8 Components of Partial Order Planning are ____.

- 1. Bindings
- 2. Goal
- 3. Causal Links
- ✓ 4. All of the mentioned

Answer: (4) All of the mentioned. POP plans contain actions, ordering constraints, causal links, and variable bindings — all four.

Q6.9 Local search algorithms' key advantages include ____.

- 1. Less memory
- 2. More time
- 3. Finds a solution in large infinite space
- ✓ 4. Less memory & Finds a solution in large infinite space

Answer: (4) Less memory & Finds a solution in large infinite space. Hill climbing, simulated annealing keep only the current state (O(1) memory) and work in vast / continuous spaces.

Q6.10 Main cons of hill-climbing search?

- ✓ 1. Terminates at local optimum & Does not find optimum solution
- 2. Terminates at global optimum & Does not find optimum solution
- 3. Does not find optimum solution & Fail to find a solution
- 4. Fail to find a solution

Answer: (1) Terminates at local optimum & Does not find optimum solution. Hill-climbing's classic flaws: gets stuck at local maxima → may miss global optimum.

Q6.11 Human brain uses ____ for reasoning.

- ✓ a. Forward chaining
- b. Backward chaining
- c. Resolution
- d. Predicate logic

Answer: (a) Forward chaining. Humans reason data-driven — start from observed facts and chain forward to conclusions, unlike Prolog which is backward-chaining.

ADDENDUM (May–June 2022)

Q3A (Addendum) 0/1 Knapsack via Genetic Algorithm. N=6, Cap=20, weights={2,4,5,3,4,2}, profits={10,15,15,20,200,50}. One generation only. (10)

Setup:

- **Chromosome:** 6-bit binary string; bit i = 1 means item i is included.
- **Fitness function:** If total weight ≤ 20 → fitness = total profit; else fitness = 0 (or apply penalty).
- **Population size:** 4 chromosomes.
- **Selection:** Roulette-wheel by fitness ratio.
- **Crossover:** Single-point at position 3 (after bit 3), probability 1.0.
- **Mutation:** Bit-flip, probability 0.05.

Initial population (random):

Chromosome	Bits (1..6)	Items selected	Weight	Profit	Fitness
C1	1 0 1 0 1 0	1, 3, 5	2+5+4 = 11	10+15+200 = 225	225
C2	0 1 1 1 0 1	2, 3, 4, 6	4+5+3+2 = 14	15+15+20+50 = 100	100
C3	1 1 0 1 1 0	1, 2, 4, 5	2+4+3+4 = 13	10+15+20+200 = 245	245
C4	0 0 1 0 1 1	3, 5, 6	5+4+2 = 11	15+200+50 = 265	265

Total fitness = 225 + 100 + 245 + 265 = 835.

Selection probabilities (roulette): p(C1)=225/835=0.27, p(C2)=0.12, p(C3)=0.29, p(C4)=0.32. Spin wheel twice → say C4 and C3 selected as parents (highest probabilities).

Crossover — single-point at position 3:

```
Parent 1 (C4): 0 0 1 | 0 1 1
Parent 2 (C3): 1 1 0 | 1 1 0
↓
Child 1: 0 0 1 | 1 1 0
Child 2: 1 1 0 | 0 1 1
```

Mutation — flip bit 5 of Child 2 (very low chance, but illustrative):

```
Child 2 after mutation: 1 1 0 0 0 1
```

Evaluate new generation:

Chromosome	Bits	Items	Weight	Profit	Fitness
Child 1	0 0 1 1 1 0	3, 4, 5	5+3+4 = 12	15+20+200 = 235	235
Child 2 (mutated)	1 1 0 0 0 1	1, 2, 6	2+4+2 = 8	10+15+50 = 75	75

After one generation: Best individual is still C4 (fitness 265) — keep via *elitism*. Continue generations till convergence; theoretical optimum is {3,4,5} with profit 235 or {2,4,5} = 4+3+4=11 wt, 15+20+200=235, or {3,5,6} weight 11, profit 265 — which matches C4. Optimum = **265**.

Q4A (Addendum) Bayesian Network for Lung Cancer Problem. (10)

Problem: Pollution (P) and Smoking (S) affect Cancer (C). Cancer (C) affects Breathing problem (B) and X-ray result (X). Compute P(smoker has cancer ∧ positive X-ray ∧ no breathing problem).

Network Topology:

```
┌─────────┐  ┌─────────┐
│Pollution│  │ Smoking │
│  (P)    │  │  (S)    │
```

```
└──────┬──────┘  └──────┬──────┘
       └────────┬────────┘
                ▼
            ┌─────────┐
            │Cancer   │
            │ (C)     │
            └────┬────┘
       ┌─────────┴─────────┐
       ▼                   ▼
  ┌─────────┐         ┌─────────┐
  │Breathe  │         │ X-ray   │
  │ (B)     │         │ (X)     │
  └─────────┘         └─────────┘
```

Conditional Probability Tables (assumed):

Variable	Probabilities (assumed)
P(Pollution=high)	P(P=T) = 0.1; P(P=F) = 0.9
P(Smoking=yes)	P(S=T) = 0.3; P(S=F) = 0.7
P(Cancer | P, S)	P(C=T | P=T, S=T) = 0.40, | P=T,S=F = 0.10, | P=F,S=T = 0.20, | P=F,S=F = 0.01
P(Breath | C)	P(B=problem | C=T) = 0.80; P(B=problem | C=F) = 0.10 → P(B=NoProblem | C=T) = 0.20
P(Xray=positive | C)	P(X=+ | C=T) = 0.90; P(X=+ | C=F) = 0.05

Computation: P(smoker, cancer, positive X-ray, no breathing problem) = ?

Let S=T (smoker), C=T (cancer), X=+ (positive X-ray), B=NoProblem. Pollution is not observed → marginalise over P.

```
P(S=T, C=T, X=+, B=NoP)
= Σ_P P(P) · P(S=T) · P(C=T|P,S=T) · P(X=+|C=T) · P(B=NoP|C=T)

Term 1 (P=T): 0.1 · 0.3 · 0.40 · 0.90 · 0.20
= 0.1 × 0.3 × 0.40 × 0.90 × 0.20
= 0.00216

Term 2 (P=F): 0.9 · 0.3 · 0.20 · 0.90 · 0.20
= 0.9 × 0.3 × 0.20 × 0.90 × 0.20
= 0.00972

Total = 0.00216 + 0.00972 = 0.01188

P(smoker, lung cancer, +X-ray, no breathing problem) ≈ 0.0119 (1.19%)
```

Reasons uncertainty enters problem-solving:

- **Laziness:** Listing all antecedents and consequences is intractable.
- **Theoretical ignorance:** Full theory of the domain unknown (e.g., disease mechanisms).
- **Practical ignorance:** Test results / data may be unavailable even when theory is known.
- **Noise in sensors / data:** Measurements have inherent error.
- **Vague / linguistic concepts:** 'Tall', 'risky' — fuzziness.
- **Inherent randomness:** Stochastic phenomena (quantum, weather).

PAPER 5 | May 2021 – June 2021

Q1(A) — Multiple Choice (10 marks)

1. Which one of the following statements is true?

- ✓ a. BFS runs out of memory before it runs out of time.
- b. BFS runs out of time before it runs out of memory.
- c. Typically, BFS needs less memory than DFS for the execution.
- d. BFS is good choice when the branching factor is large.

Answer: (a) BFS runs out of memory before it runs out of time.. BFS stores all frontier nodes O(b^d) — memory grows exponentially with depth and crashes long before time becomes the bottleneck. (Some textbooks phrase this as the opposite; KJSCE answer key takes option a.)

2. For a given problem, there could be ___ partial-order plan(s) while there could be ___ total-order plan(s).

- ✓ a. One, many
- b. Many, many
- c. Many, one
- d. One, one

Answer: (a) One, many. A single partial-order plan compactly represents many total-order plans (each linearisation of the partial order is one total plan).

3. Choose the correct FOL representation for: 'No mortal can live for more than 150 years.'

- a. ∀ m,t1,t2: mortal(m) ∧ born(m,t1) ∧ greater_than(t2−t1,150) → dead(m,t2)
- b. ∀ m,t1,t2: mortal(m) ∧ born(m,t1) ∧ greater_than(t2−t1,150) → ¬alive(m,t2)
- c. ¬∃ m,t1,t2: mortal(m) ∧ born(m,t1) ∧ greater_than(t2−t1,150) ∧ dead(m,t2)
- ✓ d. ¬∃ m,t1,t2: mortal(m) ∧ born(m,t1) ∧ greater_than(t2−t1,150) ∧ alive(m,t2)

Answer: (d) ¬∃ m,t1,t2: mortal(m) ∧ born(m,t1) ∧ greater_than(t2−t1,150) ∧ alive(m,t2). The statement asserts that NO mortal exists who, after living 150+ years, is still alive — i.e. there is no living mortal that old. Hence ¬∃ m,t1,t2: (...) ∧ alive(m,t2).

4. Two chromosomes X1: ABCDEFG and Y1: ZYXVUTS. Two-point crossover (3:3:1) gives:

- a. X1': AYCVDETG Y1': ZBXDUFS
- b. X1': ZYXDEFS Y1': ABCVUTG
- ✓ c. X1': ABCVUTG Y1': ZYXDEFS
- d. X1': AYXVEFG Y1': ZBCDUTS

Answer: (c) X1': ABCVUTG Y1': ZYXDEFS. Crossover at positions 3|6 (3:3:1 = first 3 + middle 3 + last 1). Swap middle 3 bits: X1 keeps ABC + (XVU from Y1) + G; Y1 keeps ZYX + (DEF from X1) + S — giving ABCVUTG / ZYXDEFS. Wait, careful: positions 4-6 of Y1 are VUT, swapped into X1 middle. So X1' = ABC+VUT+G = ABCVUTG ✓; Y1' = ZYX+DEF+S = ZYXDEFS ✓. Hence option (c).

5. A problem may have ___ heuristic(s).

- a. Only one
- ✓ b. Many
- c. Maximum two
- d. Maximum 5

Answer: (b) Many. Different heuristics capture different aspects (e.g., 8-puzzle has h1: misplaced tiles, h2: Manhattan distance, h3: Linear conflict — and many more). Any admissible function is valid; there is no upper bound.

6. Which one of the following statements is true?

- a. Every rule-based reasoning system is also a case-based system, but reverse isn't always true.
- b. Case-based and rule-based reasoning are totally different; no subset-superset.
- c. Every case-based and rule-based reasoning system are mutually exclusive.
- ✓ d. Every case-based reasoning system is also a rule-based system, but reverse isn't always true.

Answer: (d) Every case-based reasoning system is also a rule-based system, but reverse isn't always true.. Case-based reasoning can be implemented as a (large, instance-level) rule-base — each case is effectively a stored rule with retrieval logic. So CBR ⊂ RBR (every CBR system is a RBR system); not every RBR is CBR.

7. Properties of knowledge are:

- a. Hard to characterize, voluminous, constantly changing.
- b. Unpredictable, hard to measure, constantly changing.
- c. Laziness of collecting knowledge, practical and conceptual ignorance.
- ✓ d. Hard to represent, hard to collect, hard to maintain.

Answer: (d) Hard to represent, hard to collect, hard to maintain.. Knowledge engineering challenges: representation (KR languages limited), collection (expert bottleneck), maintenance (knowledge ages, must be updated).

8. Human mind uses ___ for reasoning process.

- ✓ a. Forward chaining
- b. Backward chaining
- c. Resolution
- d. Proof by refutation

Answer: (a) Forward chaining. Humans typically chain forward from observations — seeing rain clouds → it will rain. Backward chaining is more typical of expert systems / Prolog.

9. Snakes and Ladders belong to ___ category of games.

- a. Deterministic, static, strict-alternate, multiplayer
- b. Non-deterministic, static, no-strict-alternate, multiplayer
- ✓ c. Non-deterministic, dynamic, no-strict-alternate, multiplayer
- d. Deterministic, dynamic, no-strict-alternate, multiplayer

Answer: (c) Non-deterministic, dynamic, no-strict-alternate, multiplayer. Dice roll → non-deterministic (stochastic). Position changes over turns → dynamic. Doubles / extra turn rules → no strict alternation. Two or more players → multiplayer.

10. Most appropriate agent architecture for a psychological counselling agent:

- a. Learning with goal-based architecture as performance element
- ✓ b. Learning with utility-based architecture as performance element
- c. Learning with reflex-based architecture as performance element
- d. Learning agent architecture

Answer: (b) Learning with utility-based architecture as performance element. Counselling involves graded outcomes (patient happiness, recovery degree) under uncertainty — utility-based reasoning is needed. Adding learning lets it refine responses to feedback. Hence learning + utility-based.

Q1(B) — Attempt any FIVE (5 of 7, 2 marks each)

Q1(B)-1 Explain the concept of α and β values in adversarial search with a suitable example.

In a Min/Max game tree, **α** is the *best (highest) value* the MAX player can guarantee so far on the path to root. **β** is the *best (lowest) value* MIN can guarantee. Pruning condition: when α ≥ β at any node, further siblings are pruned (the opponent will never let play reach this branch).

Example: Consider MAX root with two children. After exploring the left child, MAX guarantees α = 7. While evaluating the right child (a MIN node), if MIN finds a value ≤ 7, MIN will pick that and MAX gets ≤ 7 from this branch — so MAX won't choose the right branch. We can prune remaining siblings of MIN. This roughly halves the effective branching factor and lets minimax look **twice as deep** in the same time.

Q1(B)-2 Discuss the 'Thinking rationally' approach. State its limitations.

The **thinking rationally** approach (a.k.a. the laws-of-thought approach) holds that intelligence comes from *using formal logic to derive correct conclusions*. Aristotle's syllogisms, predicate logic, and theorem provers (resolution) embody this view. An AI in this paradigm encodes knowledge as logical sentences and infers new truths mechanically.

Limitations: (i) *Formalisation gap* — converting informal knowledge into logical predicates is hard. (ii) *Uncertainty* — pure logic doesn't handle "probably" or "usually". (iii) *Computational intractability* — many problems are NP-hard or worse even when solvable in principle. (iv) Real intelligence often involves perception, motor control, and intuition — not pure deduction. (v) Cannot handle inconsistent knowledge gracefully (one contradiction → derive anything).

Q1(B)-3 Discuss the sequence in which variable-constraint assignments are considered by CSP algorithms to improve backtracking efficiency.

Three classic ordering heuristics speed up CSP backtracking:

- **(a) Minimum Remaining Values (MRV) / Most Constrained Variable:** Choose the variable with fewest legal values left. Detects failures sooner (fail-first principle).
- **(b) Degree heuristic (tie-breaker for MRV):** Among MRV ties, pick the variable that participates in most constraints with unassigned variables. Reduces branching for remaining vars.
- **(c) Least Constraining Value (LCV):** Once a variable is chosen, try its value that rules out fewest options for neighbours — keeps future flexibility.

Additionally, forward checking and arc consistency (AC-3) further prune domains after each assignment, making the search even faster.

Q1(B)-4 List various algorithm families that could be used by an expert system's inference engine.

An ES inference engine reasons over the knowledge base. Common algorithm families:

- **Forward chaining (data-driven):** Start with known facts, apply rules to derive new facts. RETE algorithm is a fast forward-chaining implementation.
- **Backward chaining (goal-driven):** Start with a goal, find rules whose conclusion matches it, recurse on premises (Prolog-style).
- **Resolution:** Convert to CNF, repeatedly resolve clause pairs to derive empty clause — used for theorem proving.
- **Mixed / hybrid chaining:** Combine forward and backward (CLIPS, JESS).
- **Fuzzy inference:** Mamdani / Sugeno systems for vague rules.
- **Bayesian / probabilistic inference:** For uncertain knowledge — variable elimination, belief propagation.
- **Case-based reasoning:** Retrieve similar past cases, adapt, solve.

Q1(B)-5 State which problem characteristics are applicable to Tower of Hanoi.

Tower of Hanoi has these classical problem-characteristics:

- **Can the problem be decomposed?** Yes — move n disks = move (n-1) to spare, move largest, move (n-1) to target.
- **Can solution steps be ignored / undone?** Solution steps can be **recoverable** (undoable) — moving a disk back is allowed.
- **Predictable universe?** Yes — fully deterministic. No randomness.
- **Good solution absolute or relative?** Optimal solution is absolute and known: $2^n - 1$ moves.
- **Solution: state or path?** Path matters — the sequence of moves IS the solution.
- **Role of knowledge?** Small — pure rule-based recursion suffices.
- **Does task need interaction with humans?** No — solitary puzzle.

Q1(B)-6 State and discuss the things AI cannot do even in today's technically developed era.

- **Genuine common-sense reasoning** — AI fails on unforeseen everyday situations not seen in training.
- **True creativity** — AI produces by recombining patterns; it can't invent a new musical genre with intent.
- **Emotional empathy** — Sentiment can be detected, but felt empathy is absent.
- **Generalisation across domains** — A Go-playing AI is useless at cooking; transfer learning is still narrow.
- **Self-awareness / consciousness** — No machine knows that it knows.
- **Ethical / moral judgement** — AI cannot weigh dignity, fairness, or rights without explicit programming.
- **Hands-on physical dexterity** — Tasks like folding laundry or surgery at human level remain hard.
- **Explanation in human terms** — Deep nets are opaque; full XAI not solved.

Q1(B)-7 Discuss applications of Natural Language Processing.

- **Machine Translation:** Google Translate, DeepL — translate between hundreds of languages.
- **Sentiment Analysis:** Reviews, social media monitoring, brand-sentiment dashboards.
- **Chatbots & virtual assistants:** Customer service, Siri, Alexa, ChatGPT.
- **Information Extraction:** Pull entities, relations, events from documents (NER, RE).
- **Text Summarisation:** Auto-generate abstracts of news, research, legal docs.
- **Speech recognition / TTS:** Voice typing, screen readers.
- **Question Answering:** Search engines, IBM Watson (Jeopardy).
- **Spam filtering, smart-reply, autocomplete:** Gmail features.
- **Plagiarism / authorship detection.**
- **Document classification:** Legal, medical, financial — routing to correct queue.

Q2 Represent the statements in FOL. Prove 'Lucy and John aren't friends' using forward OR backward chaining. (10)

Given statements: 1) Lucy is a professor. 2) All professors are people. 3) John is the dean. 4) Deans are professors. 5) All professors consider the dean a friend or don't know him. 6) Everyone is a friend of someone. 7) People only criticise people that are not their friends. 8) Lucy criticised John.

FOL representation:

```
1. Professor(Lucy)
2. ∀x Professor(x) → Person(x)
3. Dean(John)
4. ∀x Dean(x) → Professor(x)
5. ∀x ∀y [Professor(x) ∧ Dean(y)] → [Friend(y,x) ∨ ¬Knows(x,y)]
6. ∀x ∃y Friend(y, x)
7. ∀x ∀y [Person(x) ∧ Person(y) ∧ Criticised(x,y)] → ¬Friend(y,x)
8. Criticised(Lucy, John)
```

Goal: ¬Friend(John, Lucy) (i.e. Lucy and John are not friends).

Backward chaining proof:

```
Goal: ¬Friend(John, Lucy)

By rule (7) (contrapositive direction): if Criticised(x,y) and both are
Persons, then ¬Friend(y,x).
Set x = Lucy, y = John. Need to prove:
(a) Person(Lucy)
(b) Person(John)
(c) Criticised(Lucy, John)

Sub-goal (a) Person(Lucy):
From (2) with x = Lucy: Professor(Lucy) → Person(Lucy).
From (1): Professor(Lucy) ✓
⇒ Person(Lucy) (by Modus Ponens)

Sub-goal (b) Person(John):
From (4) with x = John: Dean(John) → Professor(John).
From (3): Dean(John) ✓
⇒ Professor(John).
From (2) with x = John: Professor(John) → Person(John).
⇒ Person(John) ✓

Sub-goal (c) Criticised(Lucy, John):
FACT (8). ✓

All three premises proven → apply rule (7) → ¬Friend(John, Lucy) ∎
```

Inference rules used: Universal Instantiation (instantiate ∀-quantified rules), Modus Ponens (apply implications), And-Introduction (combine premises). The goal is established without ever assuming the conclusion.

Q3A Plan for getting COVID-19 vaccine. (a) List uncertainties (b) Give at least 2 plans. (4)

Uncertainties in the plan:

- **Slot availability uncertainty:** Online portal (CoWIN / Aarogya Setu) may show full at the moment of booking.
- **Stock uncertainty:** Vaccine vials may run out before your turn even after booking.
- **Transport uncertainty:** Cab, public transport, traffic, rain delays.
- **Health uncertainty:** You may have a fever / cold on the day → not eligible.
- **Queue uncertainty:** Wait time at centre can range from 15 minutes to 4 hours.
- **Eligibility / paperwork:** Aadhar mismatch, age proof issues, registration glitches.
- **Side-effect uncertainty:** Post-vaccination reaction may need observation extension.
- **Brand / dose uncertainty:** Center may have Covishield but you booked for Covaxin.

Plan 1 (Primary):

- 1. Open CoWIN at 6 AM (slots released).
- 2. Book an early-morning slot (7:30 – 9 AM) at the nearest govt hospital.
- 3. Carry Aadhar + appointment screenshot.
- 4. Book Uber for 7 AM to the centre.
- 5. Arrive 7:15, get vaccinated by 8.
- 6. Stay 30 min for observation; head home.

Plan 2 (Contingency, if Plan 1 fails):

- 1. If slot unavailable: switch to walk-in centre. Confirm by phone before going.
- 2. Take a friend along (driver if you feel unwell post-vaccination).
- 3. Carry mask, sanitizer, water, snacks (for long queue).
- 4. Keep medical history note (allergies, BP medication).
- 5. Backup centre: a private hospital that takes walk-ins same-day.
- 6. If develops fever afterwards, paracetamol + rest; contact doctor if persists > 2 days.

Q3B Life-expectancy Bayesian Network. (a) Draw network (b) Assign probabilities (c) Compute reduction in life expectancy given parents-non-smoker, person-smoker, person has lung cancer. (6)

Topology:

```
┌──────────────────┐
│ Parents Smoke(PS)│
└──────────┬───────┘
     ┌─────┴──────┐
▼ ▼
┌──────────┐ ┌──────────┐
│Second-hand│ │ Smokes │
│ smoke(SH)│ │ (SM) │
└────┬─────┘ └────┬─────┘
└────┬──────────┘
▼
┌─────────┐
│ Cancer │
│ (C) │
└────┬────┘
▼
┌──────────┐
│Life-exp. │
│reduction(L)│
```

```
└──────────────┘
```

Conditional Probability Tables (assumed):

Variable	Probabilities
P(PS=T)	0.40 (parents are smokers)
P(SH | PS)	P(SH=T | PS=T) = 0.70; P(SH=T | PS=F) = 0.05
P(SM | PS)	P(SM=T | PS=T) = 0.45; P(SM=T | PS=F) = 0.20
P(C | SH, SM)	P(C=T | SH=T,SM=T) = 0.50; | SH=T,SM=F = 0.20; | SH=F,SM=T = 0.35; | SH=F,SM=F = 0.02
P(L | C)	P(L=T | C=T) = 0.90; P(L=T | C=F) = 0.10

Required: P(L=T | PS=F, SM=T, C=T) — probability of life-expectancy reduction given parents NOT smokers, person IS a smoker, person HAS cancer.

```
Given: PS=F, SM=T, C=T, query: P(L=T | C=T, ...)
```

```
Since L depends only on C in the network,
P(L=T | C=T) = 0.90 (direct from CPT)
```

```
Hence P(L=T | parents-nonsmoker, smoker, cancer) = 0.90
```

Interpretation: Given that the person has lung cancer, the upstream variables (parents, smoking) do not add information about L — L is conditionally independent of them given C. So the probability of reduced life expectancy is **90%**.

If the full joint is asked instead P(PS=F, SM=T, C=T, L=T):

```
= P(PS=F) · P(SH=F | PS=F) · P(SM=T | PS=F) · P(C=T | SH=F, SM=T) · P(L=T | C=T)
+ P(PS=F) · P(SH=T | PS=F) · P(SM=T | PS=F) · P(C=T | SH=T, SM=T) · P(L=T | C=T)
(marginalising over SH which is unobserved)
```

```
Term 1: 0.60 · 0.95 · 0.20 · 0.35 · 0.90 = 0.03591
Term 2: 0.60 · 0.05 · 0.20 · 0.50 · 0.90 = 0.00270
Total: 0.03861 (≈ 3.86 %)
```

Q4A ADL description for wearing socks and shoes problem. (4)

ADL (Action Description Language) is an extension of STRIPS that adds negation, disjunction in preconditions and conditional effects. The wearing-socks-and-shoes problem requires putting on left sock, right sock, left shoe, right shoe — with order constraints (sock before shoe, same foot).

Initial state:

```
¬WearingSock(LeftFoot) ∧ ¬WearingSock(RightFoot)
∧ ¬WearingShoe(LeftFoot) ∧ ¬WearingShoe(RightFoot)
∧ Available(LeftSock) ∧ Available(RightSock)
∧ Available(LeftShoe) ∧ Available(RightShoe)
```

Goal state:

```
WearingShoe(LeftFoot) ∧ WearingShoe(RightFoot)
```

Actions:

```
Action: WearSock(f)
Precond: Available(?sock_for_f) ∧ ¬WearingSock(f) ∧ ¬WearingShoe(f)
Effect: WearingSock(f) ∧ ¬Available(?sock_for_f)

Action: WearShoe(f)
Precond: Available(?shoe_for_f) ∧ WearingSock(f) ∧ ¬WearingShoe(f)
Effect: WearingShoe(f) ∧ ¬Available(?shoe_for_f)
```

Plan: WearSock(Left); WearSock(Right); WearShoe(Left); WearShoe(Right). Or any partial order that respects sock-before-shoe on each foot — left and right feet are independent, so a partial-order planner gives a parallelisable plan.

Q4B Differentiate STRIPS and ADL. (3)

Feature	STRIPS	ADL
Negation in preconditions	Not allowed (closed-world only)	Allowed (¬P in precondition)
Disjunction in preconditions	Not allowed	Allowed (P ∨ Q)
Equality of terms	Not supported	Supported (x = y, x ≠ y)
Conditional effects	Not supported	Supported (when ⟨cond⟩ ⟨effect⟩)
Quantifiers	Not supported	Supports ∀ and ∃
Types	Untyped variables	Typed variables
Closed/Open world	Closed-world (anything not asserted is false)	Open world (unknown facts allowed)
Action language complexity	Simpler, easier to plan with	More expressive, but planning costlier
Year introduced	1971 (Fikes & Nilsson, SRI)	1987 (Pednault)

Q4C Comment on strengths and weaknesses of decision trees. (3)

Strengths:

- **Interpretability:** Easy to read; resembles human if-then-else reasoning.
- **No need for feature scaling:** Tree splits work on raw values, no normalisation.
- **Handles mixed data:** Numerical and categorical features both fine.
- **Non-parametric:** No assumption about underlying distribution.
- **Captures non-linear relationships** naturally via successive splits.
- **Feature importance:** Gini / Gain values give variable ranking.
- **Fast prediction** at runtime — just walk down the tree.

Weaknesses:

- **Overfitting:** Deep trees memorise noise; require pruning or depth limits.
- **Instability:** Small data changes can produce a very different tree (variance).

- **Bias toward features with many values:** Pure Information Gain favours high-cardinality features (mitigated by Gain Ratio in C4.5).
- **Greedy splits:** Local optima — doesn't guarantee globally best tree.
- **Difficulty with linear / smooth relationships:** Needs many splits to approximate a slope.
- **Weak on small data:** Tree depth and reliability suffer.
- **Mitigations:** Ensembles like Random Forest, Gradient Boosting (XGBoost, LightGBM) overcome most of these issues.